

Projekt	Usługi sieciowe w biznesie	
Temat	System obsługi zamówień z dostawą	 POLITECHNIKA RZESZOWSKA im. IGNACEGO ŁUKASIEWICZA
Kierunek / Grupa	Inżynieria i analiza danych	Grupa P4
Data	12 czerwca 2026	
Repozytorium	https://github.com/uswb-projekt/megabyte	

Organizacja projektu

1. Grupa projektowa jest zespołem deweloperskim pracującym zwinnie.
2. W zespole są reprezentowane poszczególne dyscypliny wytwórcze: analityk, deweloper, tester.
3. Produktem pracy zespołu jest działające i udokumentowane oprogramowanie.
4. Produkty analityka: wstępny opis wymagań, dokumentacja odkrywania encji, analityczny model dziedziny w postaci diagramu UML, wyjaśnienie modelu, dokumentacja odkrywania przypadków użycia, katalog przypadków użycia w postaci diagramu UML, opis odpowiedzialności przypadków użycia.
5. Produkty dewelopera: kod programu w stylu DDD, technologie: Java, Spring, JPA, H2, REST, Swagger.
6. Produkty testera: kolekcja testów w narzędziu Bruno dla wszystkich zaimplementowanych usług REST, baza danych H2 po wykonaniu testów.
7. Produkt wspólny: opis projektu (Word lub Markdown).
8. Rozmiar modelu dziedziny: 20 encji.
9. Liczba zamodelowanych agregatów: 3, w tym jeden składający się z >1 encji.
10. Liczba opisanych i zaimplementowanych przypadków użycia: 10.
11. Dziedzinę (biznes) ustala zespół.
12. Wszystkie artefakty są składowane w repozytorium Git.
13. Data prezentacji: 16/17.06.2026.

Spis treści

1. WPROWADZENIE.....	4
2. ANALIZA.....	5
2.1. OPIS WYMAGAŃ.....	5
2.2. MODEL DZIEDZINY	6
3.2.1. Odkrywanie pojęć.....	6
3.2.2. Szkic modelu dziedziny.....	7
3.2.3. Wyjaśnienie modelu	7
2.3. MODEL PRZYPADKÓW UŻYCIA.....	8
3. PODZIAŁ NA KONTEKSTY	11
3.1. DZIEDZINA.....	11
3.2. PRZYPADKI UŻYCIA	16
4.2.1. Kontekst: Autentykacja.....	16
4.2.2. Kontekst: Dostarczanie	17
4.2.3. Kontekst: Marketing.....	18
4.2.4. Kontekst: Oferta	19
4.2.5. Kontekst: Płatności	20
4.2.6. Kontekst: Reklamacje.....	20
4.2.7. Kontekst: Restauracje	21
4.2.8. Kontekst: Zamówienia.....	22
4. PRACE DEWELOPERSKIE	24
4.1. CEL PRACY.....	24
4.2. AGREGATY I ENCJE	24
5.2.1. Wspólna Klasa Bazowa (BaseEntity).....	24
5.2.2. Agregat Restaurant (Menu i Oferta)	25
5.2.3. Agregat FoodOrder (Zamówienie Klienta).....	25
5.2.4. Agregat Payment (Rozliczenia Finansowe)	26
5.2.5. Diagram Klas	26
4.3. LOGIKA BIZNESOWA I PRZYPADKI UŻYCIA.....	27
5.3.1. Przebieg Przypadków Użycia.....	27
5.3.2. Biznesowe Maszyny Stanów	28
5.3.3. Walidacja danych wejściowych.....	28
4.4. WARSTWA REST API (ENDPOINTY)	29
5.4.1. Wykaz wszystkich Endpointów (Swagger UI)	30
5.4.2. Spójne obsługiwane kodów błędów.....	31
4.5. URUCHOMIENIE BAZY DANYCH H2, SEEDER DANYCH ORAZ DOKUMENTACJA SWAGGER	32
5.5.1. Baza danych H2 oraz automatyczna inizjalizacja (Database Seeding).....	32

5.5.2.	Swagger UI.....	33
5.	TESTOWANIE SYSTEMU.....	34
5.1.	CEL TESTÓW	34
5.2.	METODYKA TESTOWANIA.....	34
5.3.	TESTOWANIE AGREGATU RESTAURANTS.....	35
6.3.1.	Testy pozytywne.....	35
6.3.2.	Testy negatywne	36
5.4.	TESTOWANIE AGREGATU FOOD ORDERS	37
6.4.1.	Testy pozytywne.....	37
6.4.2.	Testy negatywne	37
6.4.3.	Testy procesowe	38
5.5.	TESTOWANIE AGREGATU PAYMENTS.....	38
6.5.1.	Testy pozytywne.....	39
6.5.2.	Testy negatywne	39
5.6.	PODSUMOWANIE WYNIKÓW TESTÓW	40
5.7.	WNIOSKI.....	41
6.	ROLE W PROJEKCIE.....	42
7.	PODSUMOWANIE.....	44
	SPIS RYSUNKÓW.....	45
	SPIS TABEL	46

1. Wprowadzenie

Celem niniejszego projektu było zaprojektowanie i wdrożenie backendu dla systemu obsługi zamówień z dostawą. Zostało stworzone kompleksowe rozwiązanie, z którego mogliby korzystać klienci zamawiający jedzenie, restauracje przygotowujące posiłki oraz kurierzy realizujący dostawy. W głównej mierze nie zależało tylko na obsłudze samego "koszyka", ale też na automatyzacji procesów logistycznych i płatności.

Praca odbyła się w zespole, dzieląc się na konkretne role: analityków, projektantów, deweloperów i testerów. Cały projekt został zrealizowany zgodnie z podejściem Domain-Driven Design (DDD). Dzięki temu tak rozbudowany system został podzielony na mniejsze, niezależne konteksty (np. zamówienia, dostawy, płatności czy ofertę), co znacznie ułatwiło późniejsze programowanie.

Niniejsza dokumentacja przedstawia pełen cykl powstawania aplikacji. Najpierw zaczyna się od dokładnej analizy wymagań i modelowania dziedziny, przez identyfikację przypadków użycia, aż po szczegółowy opis prac deweloperskich z wykorzystaniem m.in. Javy, frameworka Spring Boot oraz bazy danych H2. Sprawozdanie zamyka raport z testów stworzonego REST API, które zostało przeprowadzone za pomocą narzędzia Bruno, aby upewnić się, że rozwiązanie działa stabilnie i poprawnie obsługuje błędy.

2. Analiza

2.1. Opis wymagań

System umożliwia składanie zamówień przez klientów na produkty oferowane przez restauracje i sklepy, a następnie obsługę procesu ich realizacji i dostawy przez kurierów. Każde zamówienie jest przypisywane do konkretnego klienta oraz restauracji/sklepu. Zawiera listę wybranych produktów wraz z ich ilością, ceną oraz ewentualnymi uwagami klienta.

Zamówienie rejestrowane jest w systemie z uwzględnieniem daty i czasu złożenia zamówienia, przewidywanego czasu realizacji, statusu zamówienia, form płatności oraz adresu dostawy. System umożliwia śledzenie statusu zamówienia na każdym etapie jego realizacji.

Każda restauracja/sklep posiada w systemie swoją kartotekę, w której przechowywane są informacje (nazwa, adres, godziny otwarcia, dane kontaktowe, lista oferowanych produktów). Produkty posiadają swoją nazwę, opis, kategorię, cenę oraz dostępność. W przypadku restauracji możliwe jest określenie wariantów produktu.

Klient systemu posiada swoją kartotekę zawierającą dane osobowe oraz kontaktowe (imię, nazwisko, numer telefonu, adres e-mail, adresy dostawy). Klient może posiadać wiele adresów dostawy oraz historię zamówień przechowywaną w systemie.

W procesie realizacji zamówienia udział bierze kurier, który jest odpowiedzialny za odbiór zamówienia z restauracji/sklepu oraz dostarczenie go do klienta. Każdy kurier posiada swoją kartotekę zawierającą dane identyfikacyjne, kontaktowe oraz informacje o statusie dostępności. Kurierowi przypisywane są zamówienia wraz z informacją o czasie odbioru oraz dostawy.

System umożliwia przypisanie zamówień do kurierów na podstawie ich dostępności oraz lokalizacji. Rejestrowane są czasy odbioru zamówienia z restauracji/sklepu oraz dostarczenia do klienta, co pozwala na analizę efektywności dostaw.

W przypadku pierwszego zamówienia klienta wymagane jest utworzenie konta w systemie poprzez wprowadzenie danych osobowych oraz adresowych. Natomiast w przypadku dodania nowej restauracji/sklepu, administrator systemu wprowadza jego dane oraz ofertę produktową.

System umożliwia również obsługę płatności za zamówienia, zarówno online, jak i przy odbiorze. Rejestrowane są informacje o płatności, takie jak kwota, metoda płatności oraz status transakcji.

2.2. Model dziedziny

3.2.1. Odkrywanie pojęć

W toku opisywania procesu identyfikacji aktorów oraz przypadków użycia zastosowano w tekście sprawozdania jednolite wyróżnienia wizualne: wszystkie encje (aktorów) oznaczono żółtym podświetleniem, natomiast przypadki użycia zapisano czerwoną czcionką.

System umożliwia **składanie zamówień** przez **klientów** na **produkty oferowane przez restauracje i sklepy**, a następnie obsługę **procesu** ich **realizacji i dostawy** przez **kurierów**. Każde zamówienie jest przypisywane do konkretnego klienta oraz restauracji/sklepu. Zawiera listę wybranych produktów wraz z ich ilością, ceną oraz ewentualnymi uwagami klienta.

Zamówienie rejestrowane jest w systemie z uwzględnieniem daty i czasu złożenia zamówienia, przewidywanego czasu realizacji, statusu zamówienia, **form płatności** oraz **adresu dostawy**. System umożliwia **śledzenie statusu zamówienia** na każdym etapie jego realizacji.

Każda restauracja/sklep **posiada w systemie swoją kartotekę**, w której przechowywane są informacje (nazwa, adres, godziny otwarcia, dane kontaktowe, **lista oferowanych produktów**). Produkty posiadają swoją nazwę, opis, kategorię, cenę oraz dostępność. W przypadku restauracji możliwe jest określenie **wariantów produktu**.

Klient systemu posiada swoją kartotekę zawierającą dane osobowe oraz kontaktowe (imię, nazwisko, numer telefonu, adres e-mail, **adresy dostawy**). Klient może posiadać **wiele adresów** dostawy oraz historię zamówień przechowywaną w systemie.

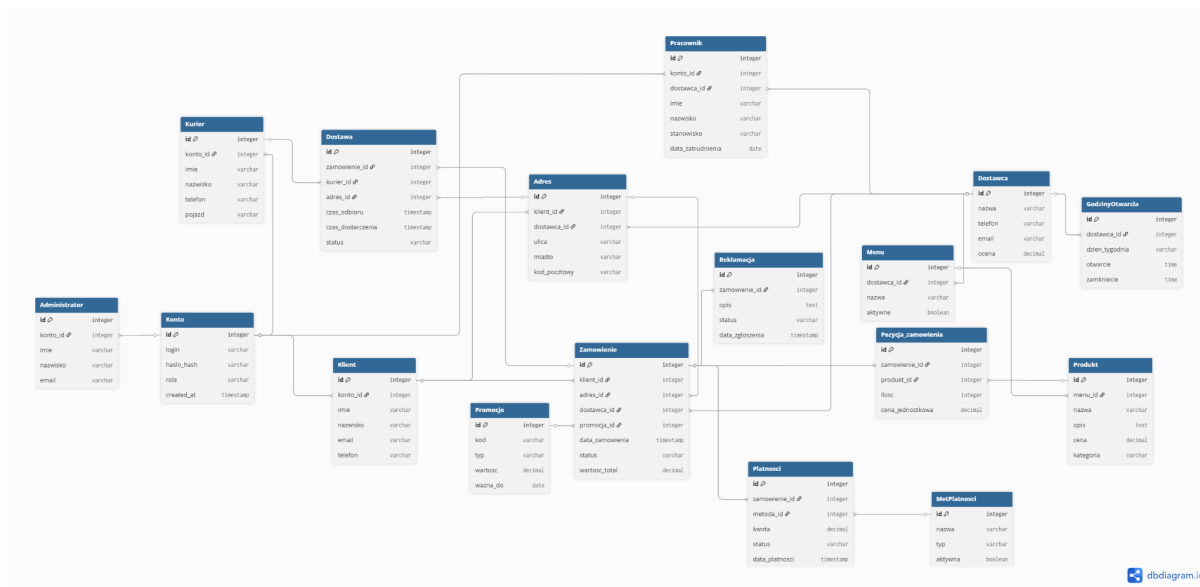
W procesie realizacji zamówienia udział bierze kurier, który jest odpowiedzialny za **odbiór zamówienia z restauracji/sklepu** oraz **dostarczenie go do klienta**. Każdy kurier posiada swoją kartotekę zawierającą dane identyfikacyjne, kontaktowe oraz informacje o statusie dostępności. Kurierowi przypisywane są zamówienia wraz z informacją o czasie odbioru oraz dostawy.

System umożliwia **przypisanie zamówień do kurierów** na podstawie ich dostępności oraz lokalizacji. Rejestrowane są czasy odbioru zamówienia z restauracji/sklepu oraz dostarczenia do klienta, co pozwala na analizę efektywności dostaw.

W przypadku pierwszego zamówienia klienta wymagane jest **utworzenie konta** w systemie poprzez **wprowadzenie danych osobowych oraz adresowych**. Natomiast w przypadku dodania nowej **restauracji/sklepu**, administrator systemu wprowadza jego dane **oraz ofertę produktową**.

System umożliwia również obsługę płatności za zamówienia, zarówno online, jak i przy odbiorze. Rejestrowane są informacje o płatności, takie jak kwota, metoda płatności oraz status transakcji.

3.2.2. Szkic modelu dziedziny



Rys. 2.1. Diagram klas - model ERD

3.2.3. Wyjaśnienie modelu

Centralnym pojęciem w rozpatrywanej dziedzinie jest zamówienie. Każde zamówienie jest składane przez konkretnego klienta, zatem w przypadku częstego korzystania z systemu, przypisana do klienta historia może zawierać wiele zamówień (relacja jeden-do-wielu od strony klienta do zamówienia). Zamówienie jest zawsze skierowane do konkretnej restauracji lub sklepu (dostawcy), przy czym dany dostawca realizuje w systemie wiele zamówień (relacja jeden-do-wielu od restauracji do zamówienia).

W ramach zamówienia rejestrowane są wybrane przez klienta produkty, co tworzy listę pozycji zamówienia (relacja jeden-do-wielu od zamówienia do pozycji zamówienia). Każda taka pozycja precyzuje wybrany produkt, jego zamawianą ilość, zamrożoną cenę oraz ewentualne uwagi (relacja jeden-do-wielu od produktu do pozycji). Produkty są przypisane do kartoteki konkretnej restauracji lub sklepu, która oferuje ich wiele (relacja jeden-do-wielu od restauracji do produktu). W przypadku restauracji produkt może dodatkowo posiadać zdefiniowane warianty (relacja jeden-do-wielu od produktu do wariantu).

Użytkownikiem zamawiającym jest klient, który posiada w systemie własną kartotekę z danymi osobowymi i kontaktowymi. Klient musi posiadać utworzone konto dostępne

(relacja jeden-do-jednego między kontem a klientem). W systemie klient może zdefiniować więcej niż jeden adres dostawy (relacja jeden-do-wielu od klienta do adresu), natomiast każde konkretne zamówienie jest powiązane z dokładnie jednym, wybranym adresem (relacja jeden-do-wielu od adresu do zamówienia).

Za fizyczną realizację procesu dostarczenia odpowiada kurier. Kurier posiada własną kartotekę określającą jego dostępność i dane kontaktowe. Może on mieć przypisanych wiele zamówień do doręczenia w ramach swojej historii pracy (relacja jeden-do-wielu od kuriera do zamówienia), przy czym konkretne zamówienie może być w danej chwili obsługiwane najwyżej przez jednego kuriera, który jest do niego przypisywany na podstawie lokalizacji (relacja zero-lub-jeden-do-wielu, gdyż nowo złożone zamówienie może jeszcze nie mieć przypisanego kuriera).

Dodatkowo, proces finalizuje płatność. Każde zamówienie posiada powiązaną ze sobą transakcję płatniczą, określającą kwotę, status oraz metodę – online lub przy odbiorze (relacja jeden-do-jednego pomiędzy zamówieniem a płatnością). Zarządzaniem całokształtem danych, w tym dodawaniem nowych kartotek restauracji i sklepów oraz ich asortymentu, zajmuje się przypisany administrator systemu.

2.3. Model przypadków użycia

Poniższy diagram oraz jego opis szczegółowo przedstawiają funkcjonalności systemu z perspektywy jego użytkowników oraz zautomatyzowanych procesów. Model identyfikuje głównych aktorów biorących udział w procesie obsługi zamówień oraz przypisane im przypadki użycia.

Zidentyfikowani Aktorzy:

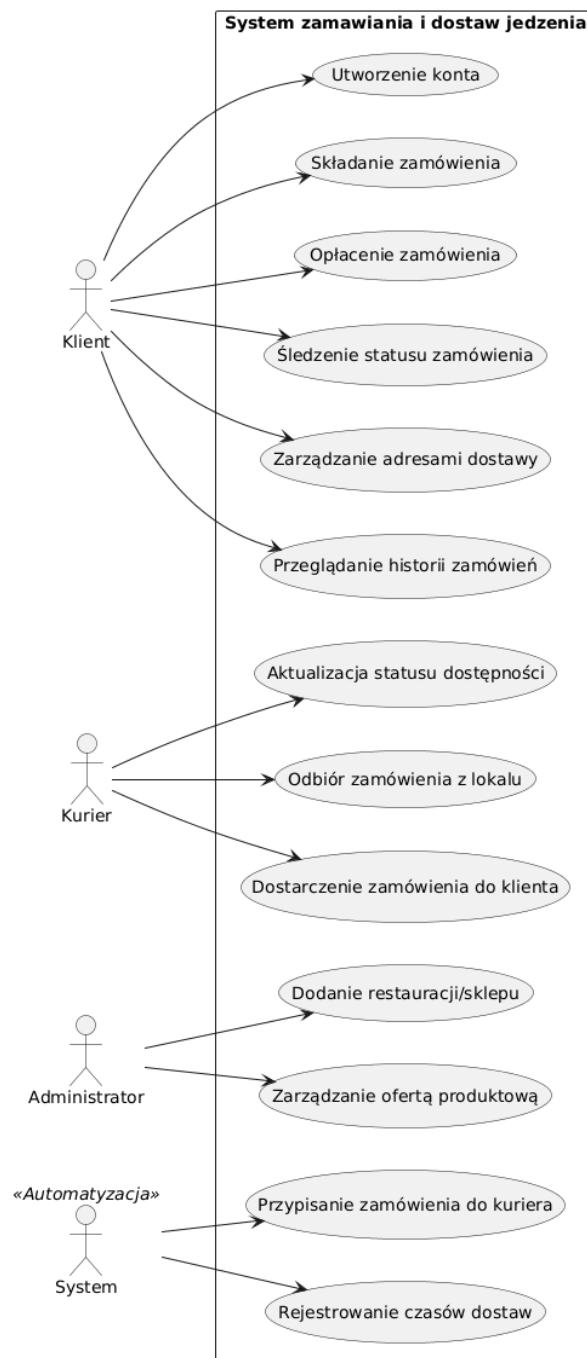
- **Klient:** Użytkownik końcowy, który składa zamówienia na produkty oferowane przez restauracje i sklepy. Posiada w systemie własną kartotekę zawierającą dane osobowe, kontaktowe, zdefiniowane adresy dostawy oraz historię zamówień.
- **Kurier:** Pracownik odpowiedzialny za fizyczną realizację procesu dostawy, czyli odbiór zamówienia z lokalu i dostarczenie go do klienta. Posiada przypisany status dostępności.
- **Administrator:** Osoba zarządzająca danymi w systemie, odpowiedzialna za wprowadzanie danych nowych restauracji i sklepów oraz zarządzanie ich ofertą produktową.

- **System** («Automatyzacja»): Aktor nieosobowy, wykonujący zautomatyzowane procesy w tle wspierające obsługę logistyczną i analityczną.

Katalog przypadków użycia (według aktorów):

- Aktor: Klient
 - **Utworzenie konta:** Wymagane w przypadku pierwszego zamówienia klienta; polega na wprowadzeniu do systemu danych osobowych oraz adresowych.
 - **Składanie zamówienia:** Główny proces biznesowy, w którym klient określa konkretną restaurację lub sklep, wybiera z listy produkty, podaje ich ilość oraz wprowadza ewentualne uwagi.
 - **Opłacenie zamówienia:** Obsługa płatności za wybrane produkty, realizowana online lub przy odbiorze.
 - **Śledzenie statusu zamówienia:** Możliwość bieżącego monitorowania przez klienta statusu zamówienia na każdym etapie jego realizacji.
 - **Zarządzanie adresami dostawy:** Możliwość dodawania i modyfikowania wielu adresów dostawy w ramach kartoteki klienta.
 - **Przeglądanie historii zamówień:** Dostęp do archiwum zrealizowanych transakcji przechowywanego w systemie.
- Aktor: Kurier
 - **Aktualizacja statusu dostępności:** Modyfikacja przez kuriera swojego statusu, na podstawie którego system weryfikuje jego gotowość do pracy.
 - **Odbiór zamówienia z lokalu:** Potwierdzenie przez kuriera fizycznego przejęcia zamówienia przygotowanego przez restaurację lub sklep.
 - **Dostarczenie zamówienia do klienta:** Sfinalizowanie procesu dostawy i przekazanie produktów klientowi.
- Aktor: Administrator
 - **Dodanie restauracji/sklepu:** Wprowadzenie do systemu kartoteki nowego dostawcy wraz z jego danymi (nazwa, adres, godziny otwarcia, dane kontaktowe).
 - **Zarządzanie ofertą produktową:** Wprowadzanie oraz edytowanie listy oferowanych produktów dla poszczególnych lokali.

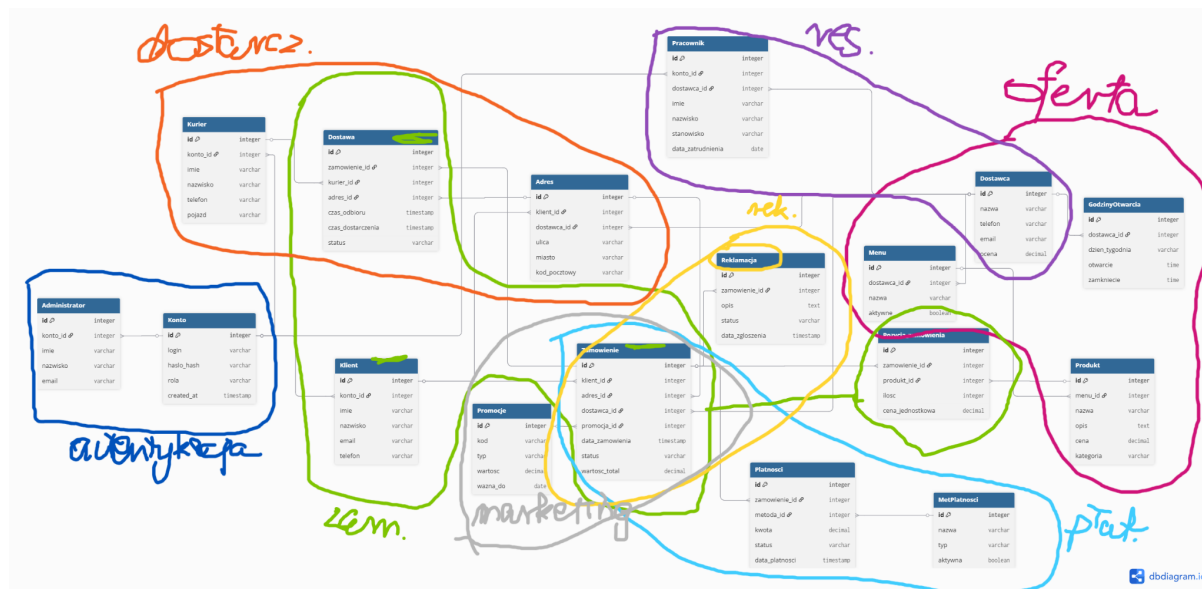
- Aktor: System («Automatyzacja»)
 - **Przypisanie zamówienia do kuriera:** Zautomatyzowany proces dobierania odpowiedniego kuriera do nowo złożonego zamówienia na podstawie jego obecnej lokalizacji oraz statusu dostępności.
 - **Rejestrowanie czasów dostaw:** Automatyczny zapis czasu odbioru zamówienia z lokalu oraz czasu dostarczenia go do klienta, co umożliwi późniejszą analizę efektywności dostaw.



Rys. 2.2. Diagram przypadków użycia - model ogólny systemu

3. Podział na konteksty

3.1. Dziedzina



Rys. 3.1. Podział na konteksty

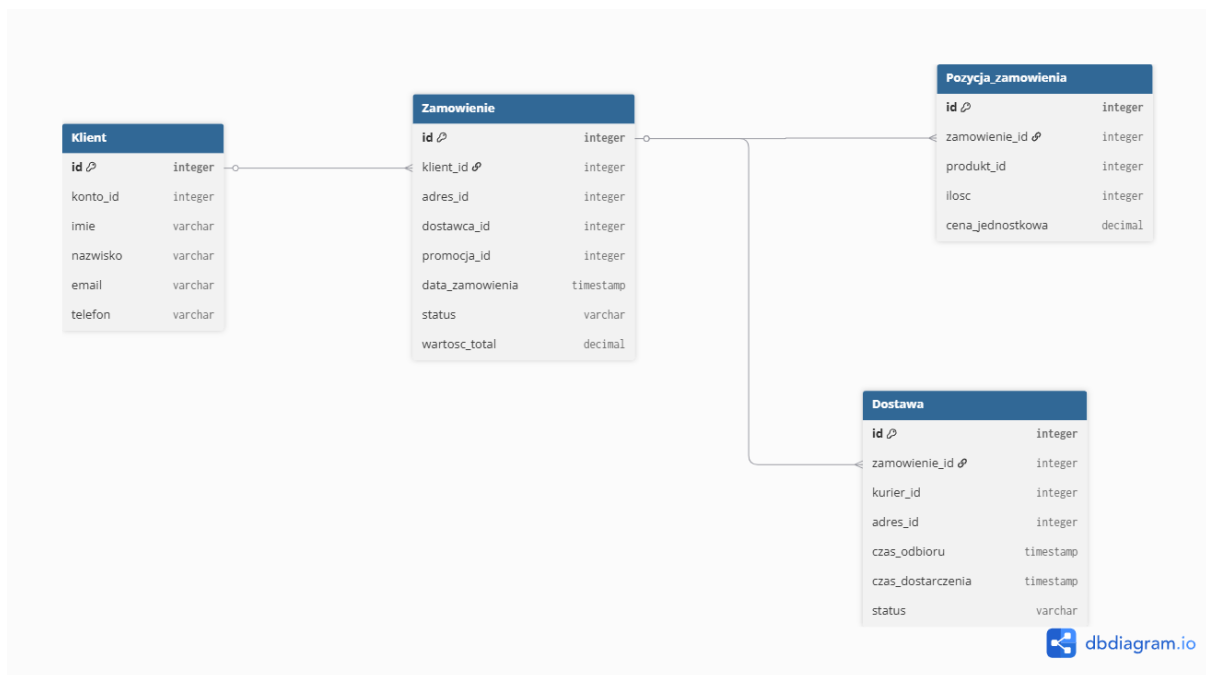
W modelu dziedziny wyodrębniono 8 kontekstów:

- **Autentykacja** - zarządzanie kontami użytkowników,
- **Dostarczanie** - logistyka realizacji dostaw,
- **Marketing** - obsługa akcji promocyjnych,
- **Oferta** - katalog dostępnych produktów,
- **Płatności** - procesowanie transakcji finansowych,
- **Reklamacje** - obsługa zgłoszeń i skarg,
- **Restauracje** - struktura wewnętrzna lokali gastronomicznych,
- **Zamówienie** - proces zakupowy.

Dziedziną główną jest kontekst Zamówienie.

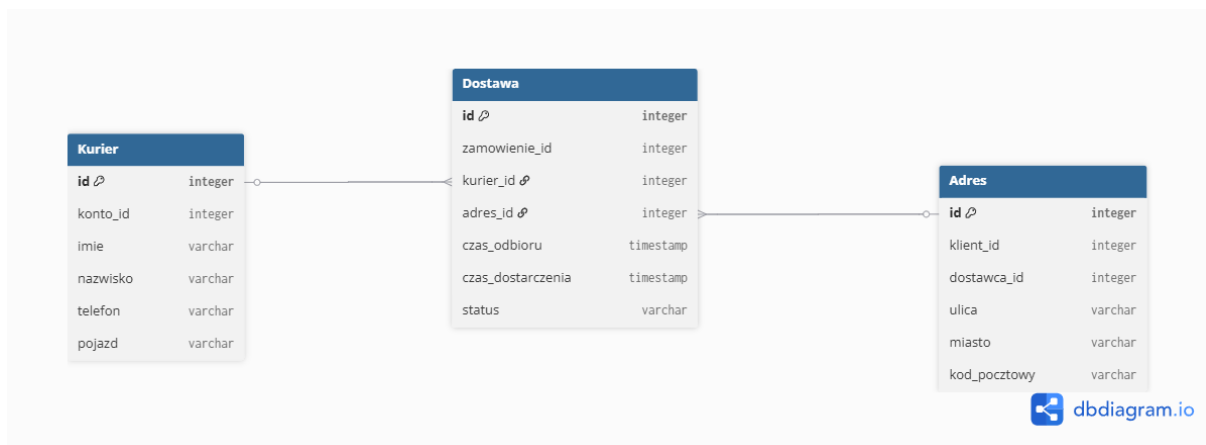
Poniżej przedstawiono następujące konteksty:

- Zamówienie:



Rys. 3.2. Kontekst Zamówienie

- Dostarczenie:



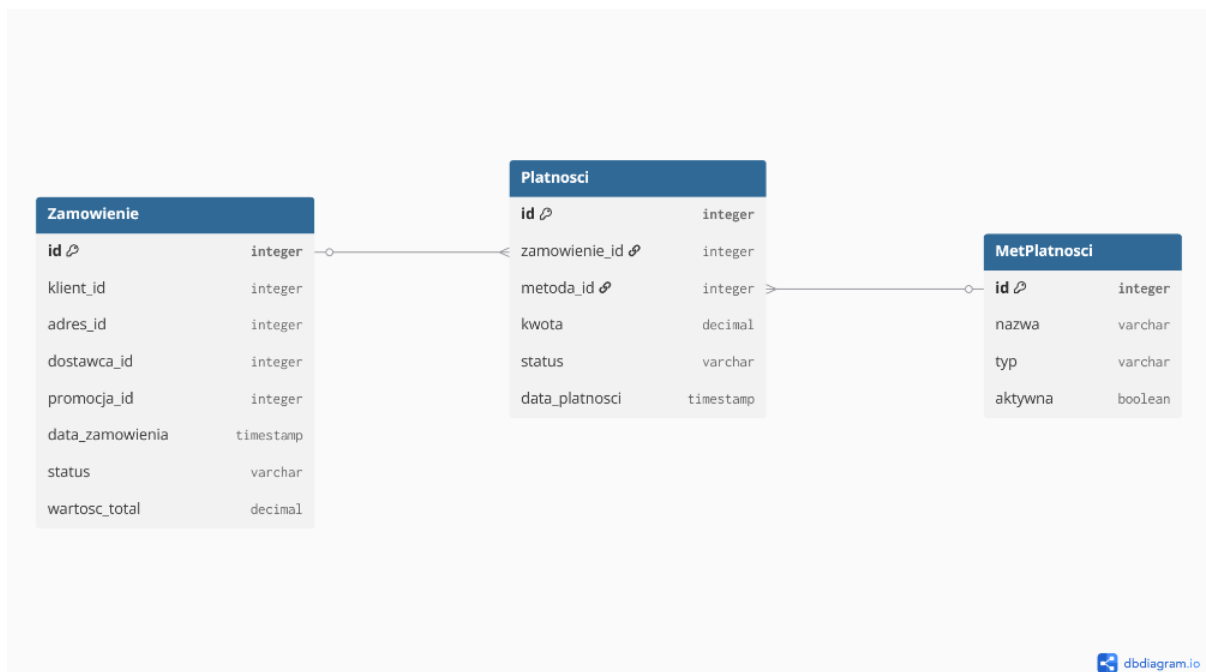
Rys. 3.3. Kontekst Dostarczenie

- Oferta:



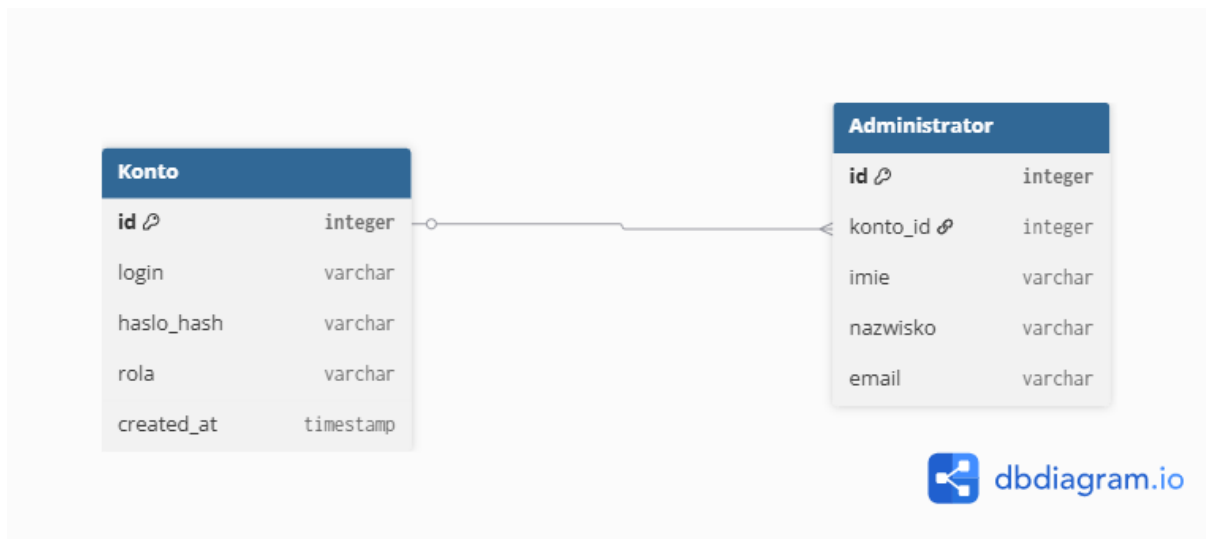
Rys. 3.4. Kontekst Oferta

- Płatności:



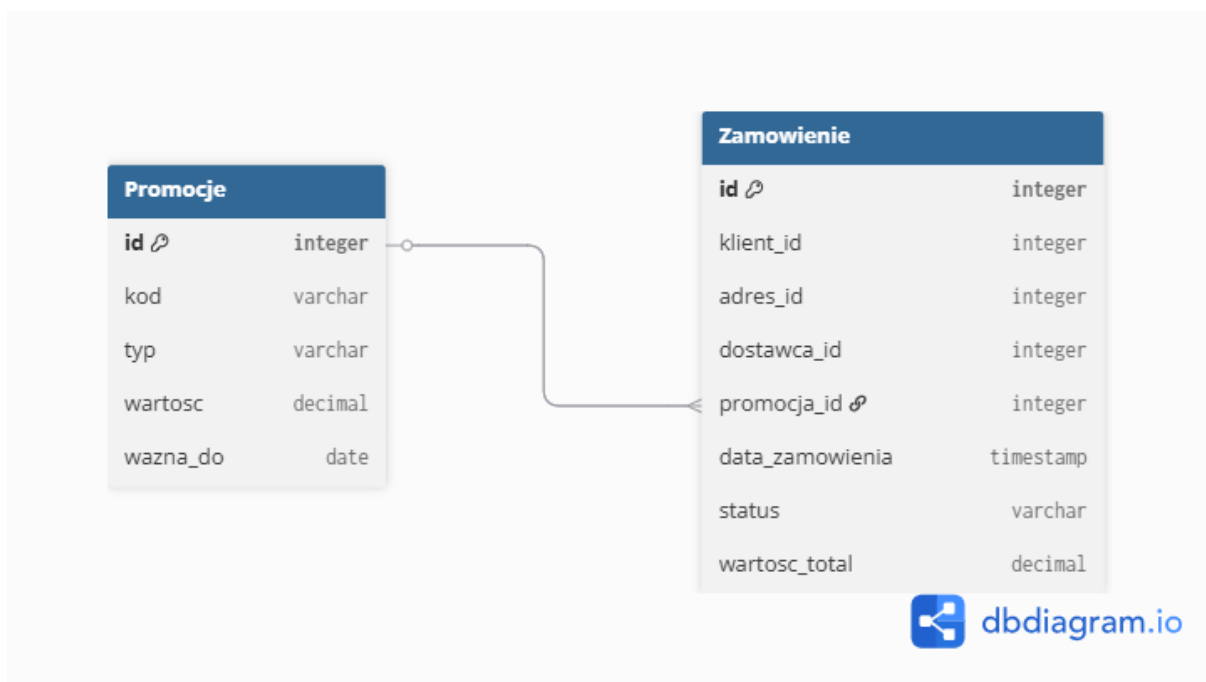
Rys. 3.5. Kontekst Płatności

- Autentykacji:



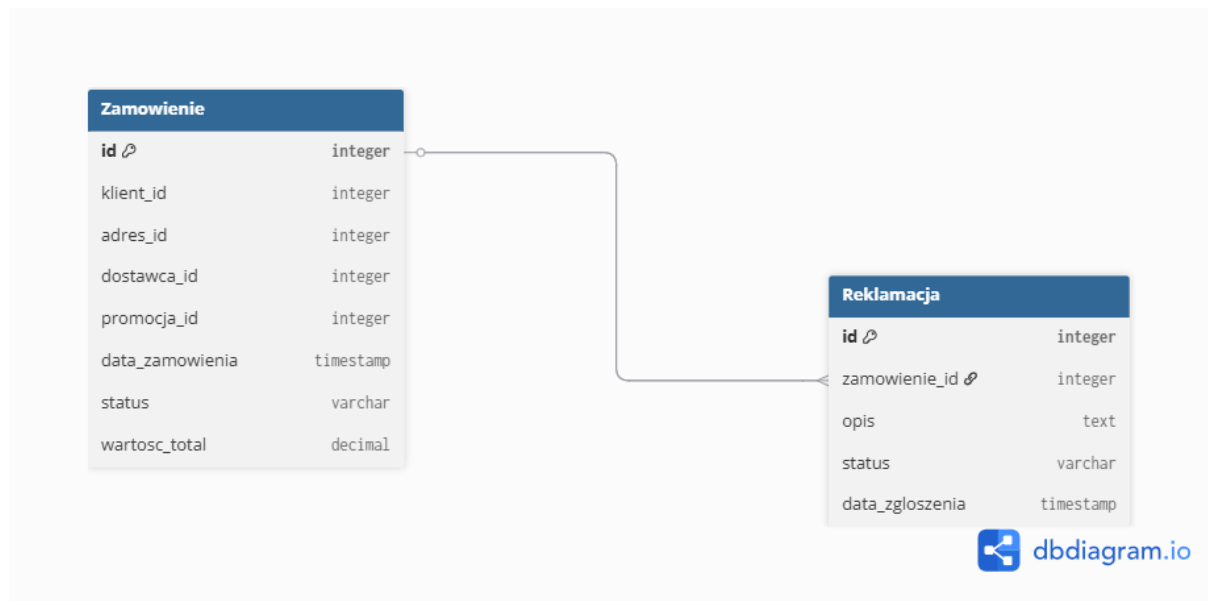
Rys. 3.6. Kontekst Autentykacji

- Marketingu:



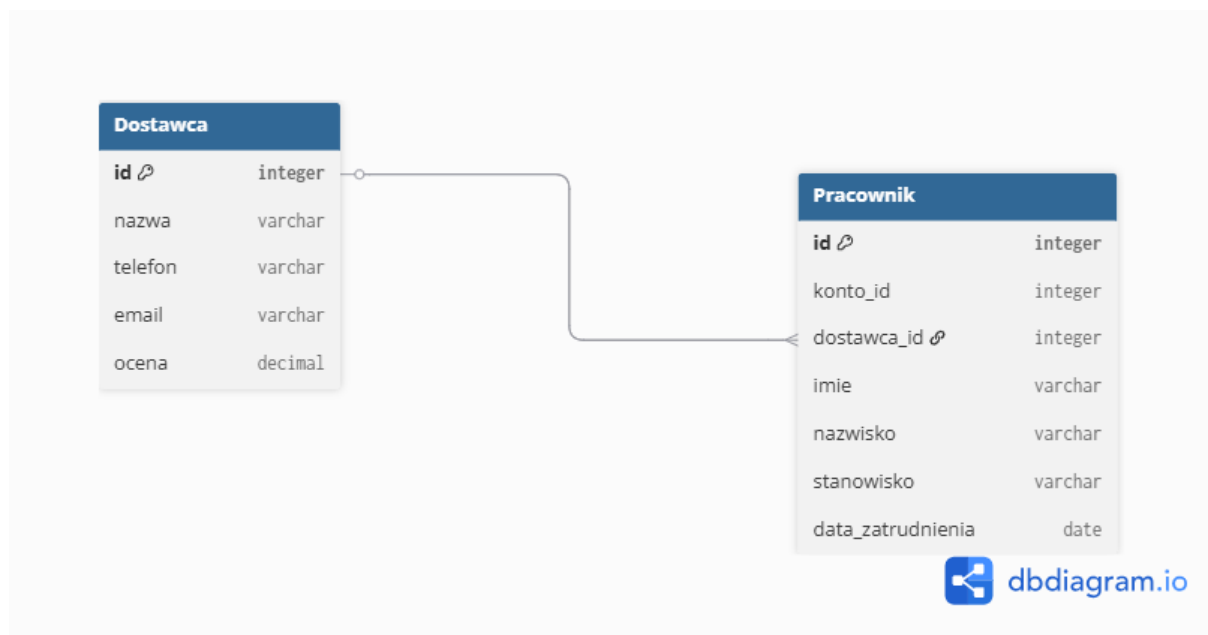
Rys. 3.7. Kontekst Marketingu

- Reklamacji:



Rys. 3.8. Kontekst Reklamacji

- Restauracji:



Rys. 3.9. Kontekst Restauracji

3.2. Przypadki użycia

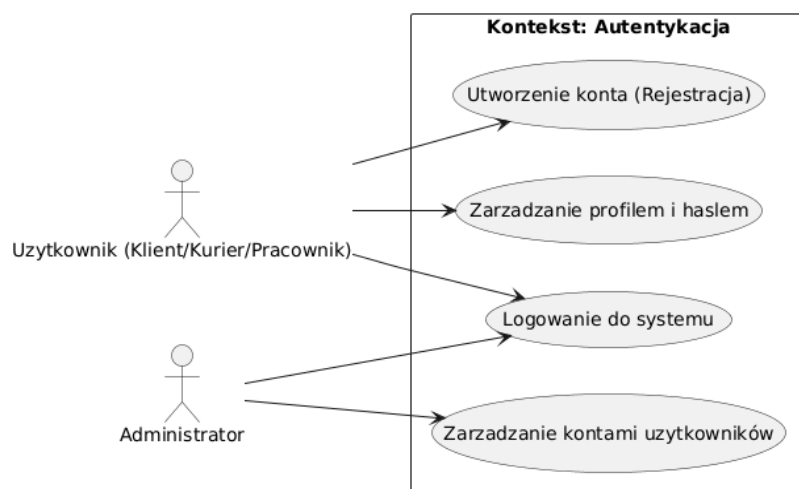
4.2.1. Kontekst: Autentykacja

Kontekst ten obejmuje przypadki użycia związane z bezpiecznym zarządzaniem tożsamością użytkowników, procesem rejestracji oraz weryfikacją uprawnień dostępowych do systemu. Zapewnia on izolację danych pomiędzy poszczególnymi rolami systemowymi.

Zaangażowani aktorzy: Użytkownik (reprezentujący role: Klient, Kurier, Pracownik) oraz Administrator.

Zidentyfikowane przypadki użycia:

- **Utworzenie konta (Rejestracja):** Przypadek użycia inicjowany przez Użytkownika (wymagany m.in. w przypadku pierwszego zamówienia klienta); polega na wprowadzeniu do systemu niezbędnych danych osobowych, kontaktowych oraz adresowych. Pozwala na założenie konta w systemie i powiązanie go z odpowiednią kartoteką.
- **Zarządzanie profilem i hasłem:** Umożliwia zarejestrowanemu Użytkownikowi samodzielną aktualizację swoich danych w przypisanej mu kartotece (np. edycję danych kontaktowych Klienta lub Kuriera) oraz dbanie o bezpieczeństwo konta poprzez zarządzanie danymi uwierzytelniającymi.
- **Logowanie do systemu:** Uniwersalny przypadek użycia wymagany od każdego aktora (zarówno Użytkownika, jak i Administratora) w celu uwierzytelnienia. Zapewnia autoryzowany dostęp do systemu, nadając uprawnienia i dostęp do funkcji zgodnych z przypisaną do konta rolą.
- **Zarządzanie kontami użytkowników:** Funkcjonalność dostępna dla Administratora, pozwalająca na globalny nadzór nad wszystkimi zarejestrowanymi w systemie kontami (np. weryfikacja, modyfikacja uprawnień ról systemowych czy rozwiązywanie problemów z dostępem).



Rys. 3.10. Diagram przypadków użycia - kontekst Autentykacja

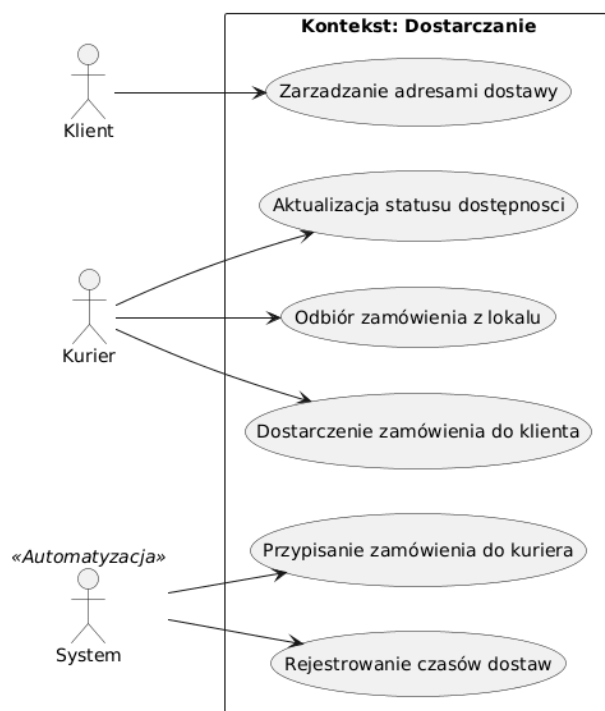
4.2.2. Kontekst: Dostarczanie

Kontekst ten skupia się na logistycznej stronie obsługi zamówień. Obejmuje zarządzanie informacjami przestrzennymi, fizyczną realizację procesu dostawy przez kurierów oraz zautomatyzowane procesy systemowe optymalizujące i monitorujące czas doręczeń.

Zaangażowani aktorzy: Klient, Kurier, System («Automatyzacja»).

Zidentyfikowane przypadki użycia:

- **Zarządzanie adresami dostawy:** Funkcjonalność przypisana Klientowi, dająca możliwość dodawania i modyfikowania wielu adresów dostawy w ramach kartoteki klienta.
- **Aktualizacja statusu dostępności:** Modyfikacja przez kuriera swojego statusu, na podstawie którego system weryfikuje jego gotowość do pracy.
- **Odbiór zamówienia z lokalu:** Potwierdzenie przez kuriera fizycznego przejęcia zamówienia przygotowanego przez restaurację lub sklep.
- **Dostarczenie zamówienia do klienta:** Sfinalizowanie procesu dostawy i przekazanie produktów klientowi.
- **Przypisanie zamówienia do kuriera:** Zautomatyzowany proces dobierania odpowiedniego kuriera do nowo złożonego zamówienia na podstawie jego obecnej lokalizacji oraz statusu dostępności.
- **Rejestrowanie czasów dostaw:** Automatyczny zapis czasu odbioru zamówienia z lokalu oraz czasu dostarczenia go do klienta, co umożliwi późniejszą analizę efektywności dostaw.



Rys. 3.11. Diagram przypadków użycia - kontekst Dostarczanie

4.2.3. Kontekst: Marketing

Kontekst ten obejmuje funkcjonalności systemu związane z kampaniami promocyjnymi oraz polityką rabatową. Skupia się na narzędziach pozwalających na zarządzanie zniżkami, mającymi na celu aktywizację użytkowników i zachęcenie ich do korzystania z platformy.

Zaangażowani aktorzy: Administrator / Marketing, Klient.

Zidentyfikowane przypadki użycia:

- **Tworzenie kodów promocyjnych:** Działanie inicjowane przez administratora lub dedykowanego pracownika działu marketingu, polegające na generowaniu i definiowaniu parametrów nowych kodów rabatowych (np. procentowych lub kwotowych) obowiązujących w systemie.
- **Zarządzanie ważnością promocji:** Funkcjonalność umożliwiająca określanie oraz modyfikację ram czasowych dla aktywnych kampanii i kodów, a także ich ewentualne przedwczesne wygasanie. Pozwala to na pełną kontrolę nad cyklem życia danej akcji marketingowej.
- **Aplikowanie kodu rabatowego:** Czynność wykonywana przez klienta, polegająca na wpisaniu ważnego kodu promocyjnego (zazwyczaj na etapie koszyka lub składania zamówienia) w celu obniżenia jego końcowej wartości.

4.2.4. Kontekst: Oferta

Kontekst ten koncentruje się na zarządzaniu katalogiem produktów oferowanych przez restauracje i sklepy w systemie. Obejmuje procesy tworzenia i definiowania asortymentu, kategoryzacji, precyzowania cen oraz udostępniania zaktualizowanego menu użytkownikom końcowym.

Zaangażowani aktorzy: Administrator / Pracownik lokalu, Klient.

Zidentyfikowane przypadki użycia:

- **Zarządzanie ofertą produktową (dodaj/usuń):** Funkcjonalność przypisana Administratorowi lub Pracownikowi lokalu, polegająca na wprowadzaniu nowych pozycji do kartoteki restauracji/sklepu oraz usuwaniu tych już wycofanych z oferty. Proces ten obejmuje określenie takich informacji jak nazwa, opis czy status dostępności produktu.
- **Zarządzanie wariantami/cenami:** Możliwość ustalania i bieżącej aktualizacji cen produktów oraz definiowania dla nich specyficznych opcji (np. warianty wielkości dania, rodzaje ciasta w pizzerii), co jest szczególnie istotne w przypadku elastycznych ofert restauracyjnych.
- **Zarządzanie kategoriami menu:** Grupowanie dostępnych produktów w logiczne kategorie (np. zupy, dania główne, napoje, desery), co pozwala na przejrzyste uporządkowanie asortymentu danego lokalu.
- **Przeglądanie dostępnego menu:** Przypadek użycia realizowany przez Klienta, pozwalający na wygodne przeglądanie aktualnej oferty wybranego lokalu, sprawdzanie dostępnych kategorii, wariantów produktów oraz ich cen przed przejściem do procesu zamówienia.



Rys. 3.12. Diagram przypadków użycia - kontekst Oferta

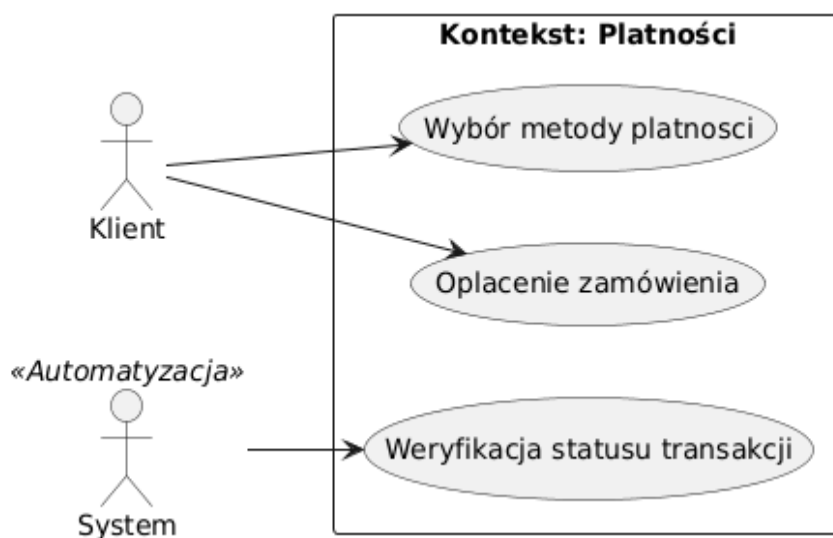
4.2.5. Kontekst: Płatności

Kontekst ten obejmuje procesy finansowe związane z finalizacją procesu zakupowego. Zapewnia on obsługę transakcji płatniczych za złożone zamówienia oraz monitorowanie i aktualizację ich statusu.

Zaangażowani aktorzy: Klient, System («Automatyzacja»).

Zidentyfikowane przypadki użycia:

- **Wybór metody płatności:** Czynność wykonywana przez klienta, polegająca na określeniu preferowanego sposobu uregulowania należności za zamówienie (np. płatność online lub przy odbiorze).
- **Oplacenie zamówienia:** Bezpośrednia realizacja transakcji przez klienta, pozwalająca na uiszczenie opłaty za wybrane produkty z oferty.
- **Weryfikacja statusu transakcji:** Zautomatyzowany proces systemowy odpowiedzialny za monitorowanie i potwierdzanie przebiegu płatności. Rejestruje on i aktualizuje informacje o danej transakcji, określając jej bieżący status.



Rys. 3.13. Diagram przypadków użycia - kontekst Płatności

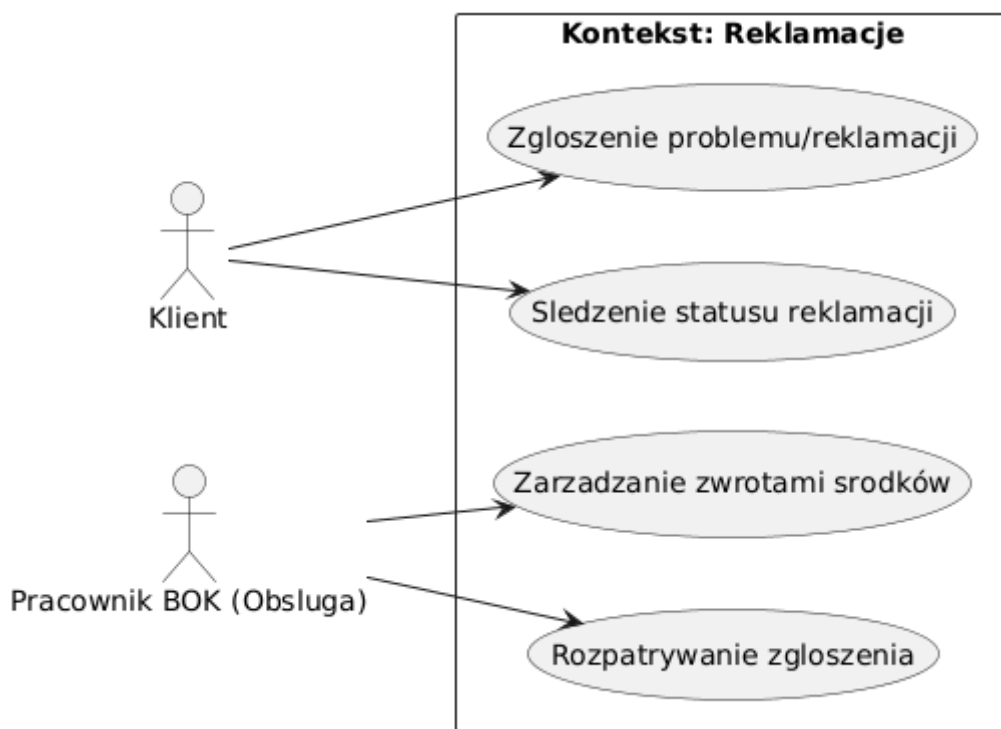
4.2.6. Kontekst: Reklamacje

Kontekst ten obejmuje procesy posprzedażowe, skupiające się na obsłudze zgłoszeń dotyczących ewentualnych problemów z realizacją zamówienia lub dostawą. Zapewnia on uporządkowaną ścieżkę komunikacji oraz narzędzia niezbędne do rozwiązywania zaistniałych niedogodności i procesowania zwrotów finansowych.

Zaangażowani aktorzy: Klient, Pracownik BOK (Obsługa).

Zidentyfikowane przypadki użycia:

- **Zgłoszenie problemu/reklamacji:** Działanie inicjowane przez klienta w sytuacji wystąpienia nieprawidłowości (np. niezgodność zamówienia, opóźnienie w dostawie, uszkodzenie produktu). Pozwala na formalne zarejestrowanie problemu w systemie.
- **Śledzenie statusu reklamacji:** Możliwość bieżącego monitorowania przez klienta postępów w rozpatrywaniu jego zgłoszenia na każdym etapie tego procesu.
- **Rozpatrywanie zgłoszenia:** Czynność wykonywana przez pracownika Biura Obsługi Klienta (BOK), polegająca na analizie zarejestrowanego problemu, weryfikacji szczegółów zamówienia oraz podjęciu decyzji o sposobie jego rozwiązania.
- **Zarządzanie zwrotami środków:** Funkcjonalność przypisana pracownikowi obsługi, umożliwiającą zlecenie i zrealizowanie zwrotu uiszczonej płatności na rzecz klienta w przypadku pozytywnego rozpatrzenia reklamacji.



Rys. 3.14. Diagram przypadków użycia - kontekst Reklamacje

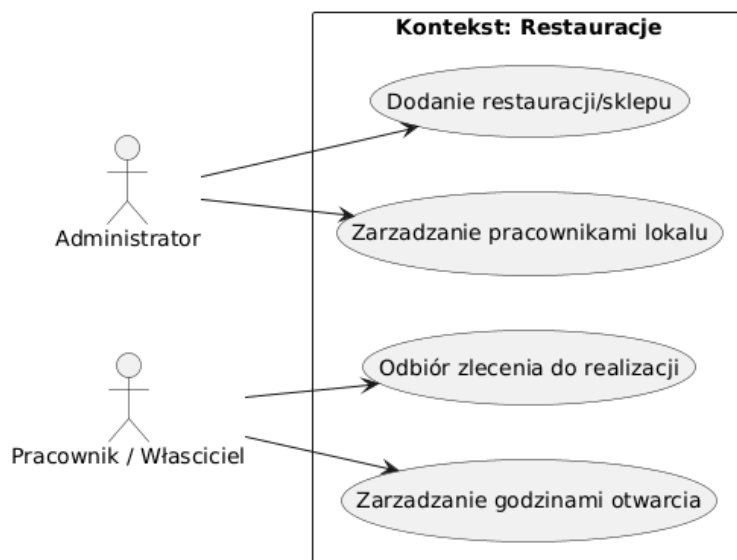
4.2.7. Kontekst: Restauracje

Kontekst ten skupia się na zarządzaniu profilami i danymi operacyjnymi dostawców usług, czyli restauracji i sklepów. Obejmuje on wprowadzanie nowych lokali do systemu, konfigurację ich parametrów działania oraz zarządzanie personelem przypisanym do konkretnego punktu.

Zaangażowani aktorzy: Administrator, Pracownik / Właściciel.

Zidentyfikowane przypadki użycia:

- **Dodanie restauracji/sklepu:** Działanie inicjowane przez Administratora, polegające na wprowadzeniu do systemu kartoteki nowego dostawcy wraz z jego danymi (nazwa, adres, godziny otwarcia, dane kontaktowe).
- **Zarządzanie pracownikami lokalu:** Funkcjonalność przypisana Administratorowi, umożliwiająca tworzenie i modyfikację kont personelu (Pracowników / Właścicieli) przypisanego do obsługi konkretnej restauracji lub sklepu.
- **Odbiór zlecenia do realizacji:** Czynność wykonywana przez pracownika lokalu, polegająca na przyjęciu spływającego z systemu zamówienia do fizycznego przygotowania w kuchni lub kompletacji w sklepie.
- **Zarządzanie godzinami otwarcia:** Umożliwia pracownikowi lub właścicielowi modyfikację i bieżącą aktualizację harmonogramu pracy lokalu, co bezpośrednio wpływa na widoczność oferty i to, czy klienci mogą w danym momencie składać zamówienia.



Rys. 3.15. Diagram przypadków użycia - kontekst Restauracje

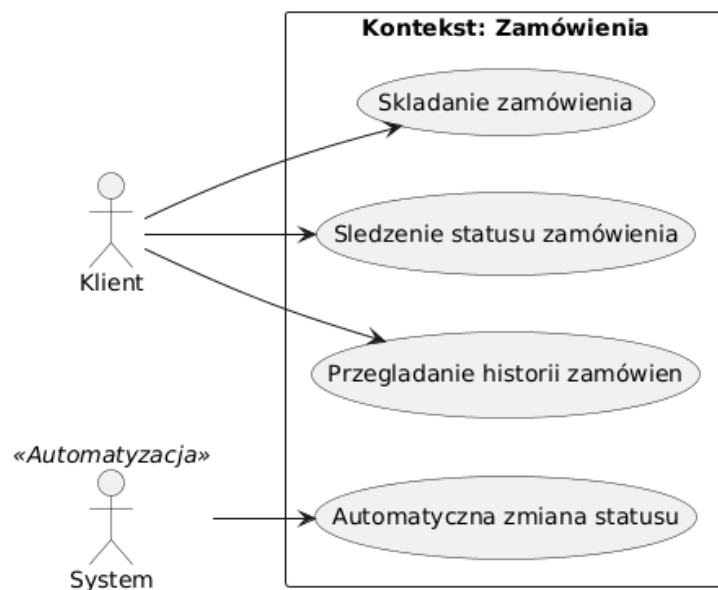
4.2.8. Kontekst: Zamówienia

Kontekst ten reprezentuje rdzeń funkcjonalności całego systemu. Skupia się na głównym procesie biznesowym, od momentu skompletowania koszyka, poprzez złożenie zamówienia, aż po jego archiwizację. Zapewnia on klientowi pełną kontrolę nad realizowanymi zakupami.

Zaangażowani aktorzy: Klient, System («Automatyzacja»).

Zidentyfikowane przypadki użycia:

- **Składanie zamówienia:** Główny proces biznesowy, w którym klient określa konkretną restaurację lub sklep, wybiera z listy produkty, podaje ich ilość oraz wprowadza ewentualne uwagi. To na tym etapie następuje związanie klienta z wybraną ofertą dostawcy.
- **Śledzenie statusu zamówienia:** Możliwość bieżącego monitorowania przez klienta statusu zamówienia na każdym etapie jego realizacji.
- **Przeglądanie historii zamówień:** Dostęp do archiwum zrealizowanych transakcji przechowywanego w systemie.
- **Automatyczna zmiana statusu:** Działanie realizowane w tle przez system, polegające na samoczynnej aktualizacji cyklu życia zamówienia (np. ze statusu "Opłacone" na "W przygotowaniu" lub "W drodze") w odpowiedzi na akcje podejmowane przez inne moduły (jak chociażby płatności czy dostarczenia).



Rys. 3.16. Diagram przypadków użycia - kontekst Zamówienia

4. Prace deweloperskie

4.1. Cel pracy

Głównym celem naszych prac programistycznych było przekształcenie prostego prototypu CRUD w bezpieczną, spójną biznesowo i gotową produkcyjnie aplikację backendową klasy Enterprise.

Cel ten został osiągnięty poprzez:

1. **Zabezpieczenie spójności danych (DDD i Walidacja):** Oddzieliliśmy bazę danych od warstwy prezentacji za pomocą klas DTO i wprowadziliśmy automatyczną walidację danych wejściowych (np. blokowanie ujemnych cen czy pustych adresów).
2. **Wdrożenie reguł biznesowych (Maszyny Stanów):** Zaimplementowaliśmy w kodzie ścisłą kontrolę statusów zamówień i płatności, co uniemożliwia wykonanie nielogicznych operacji (np. modyfikację zamkniętego zamówienia).
3. **Ujednolicenie komunikacji (REST API i Obsługa Błędów):** Stworzyliśmy czytelne endpointy oraz centralny system obsługi błędów, dzięki czemu frontend lub testerzy zawsze otrzymują przewidywalne i sformatowane odpowiedzi HTTP (200, 201, 400, 404, 409).

4.2. Agregaty i encje

Struktura danych została zorganizowana wokół agregatów DDD. Dostęp do encji wewnętrznych jest kontrolowany przez korzenie agregatów (Aggregate Roots), co gwarantuje spójność stanu systemu.

5.2.1. Wspólna Klasa Bazowa (BaseEntity)

Wszystkie encje dziedziczą po technicznej klasie bazowej BaseEntity.java:

- **id (@Id):** Klucz główny generowany automatycznie przy użyciu strategii tożsamościowej (IDENTITY).
- **version (@Version):** Pole techniczne służące do implementacji blokowania optymistycznego (Optimistic Locking). Hibernate weryfikuje tę wersję przy każdym zapisie, zapobiegając nadpisaniu danych w przypadku jednoczesnej modyfikacji rekordu przez różnych użytkowników.


```

BaseEntity.java
1  package edu.prz.delivery.foundation.domain;
2
3  import jakarta.persistence.GeneratedValue;
4  import jakarta.persistence.GenerationType;
5  import jakarta.persistence.Id;
6  import jakarta.persistence.MappedSuperclass;
7  import jakarta.persistence.Version;
8  import lombok.Data;
9  import lombok.EqualsAndHashCode;
10
11  @MappedSuperclass
12  @Data
13  @EqualsAndHashCode(of = "id")
14  public abstract class BaseEntity {
15
16      @Id
17      @GeneratedValue(strategy = GenerationType.IDENTITY)
18      private Long id;
19
20      @Version
21      private int version;
22  }

```

5.2.2. Agregat Restaurant (Menu i Oferta)

Agregat ten odpowiada za definicję oferty kulinarnej lokali.

- **Restaurant (Korzeń Agregatu):** Reprezentuje punkt gastronomiczny. Posiada kaskadową relację `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` do dań w menu.
- **Product (Encja wewnętrzna):** Reprezentuje pojedynczą pozycję w menu (nazwa, opis, cena bazowa, kategoria). Posiada relację `@OneToMany` do swoich wariantów.
- **ProductVariant (Encja wewnętrzna):** Reprezentuje konkretną konfigurację dania (np. rozmiar pizzy) z przypisaną do niej dopłatą/ceną.

5.2.3. Agregat FoodOrder (Zamówienie Klienta)

Agregat zarządzający procesem zakupowym.

- **FoodOrder (Korzeń Agregatu):** Przechowuje dane adresowe, identyfikator klienta, restauracji, kuriera oraz czas złożenia, szacowanej i rzeczywistej dostawy. Posiada

relację kaskadową `@OneToMany` do pozycji zamówienia oraz relację `@OneToOne` z płatnością.

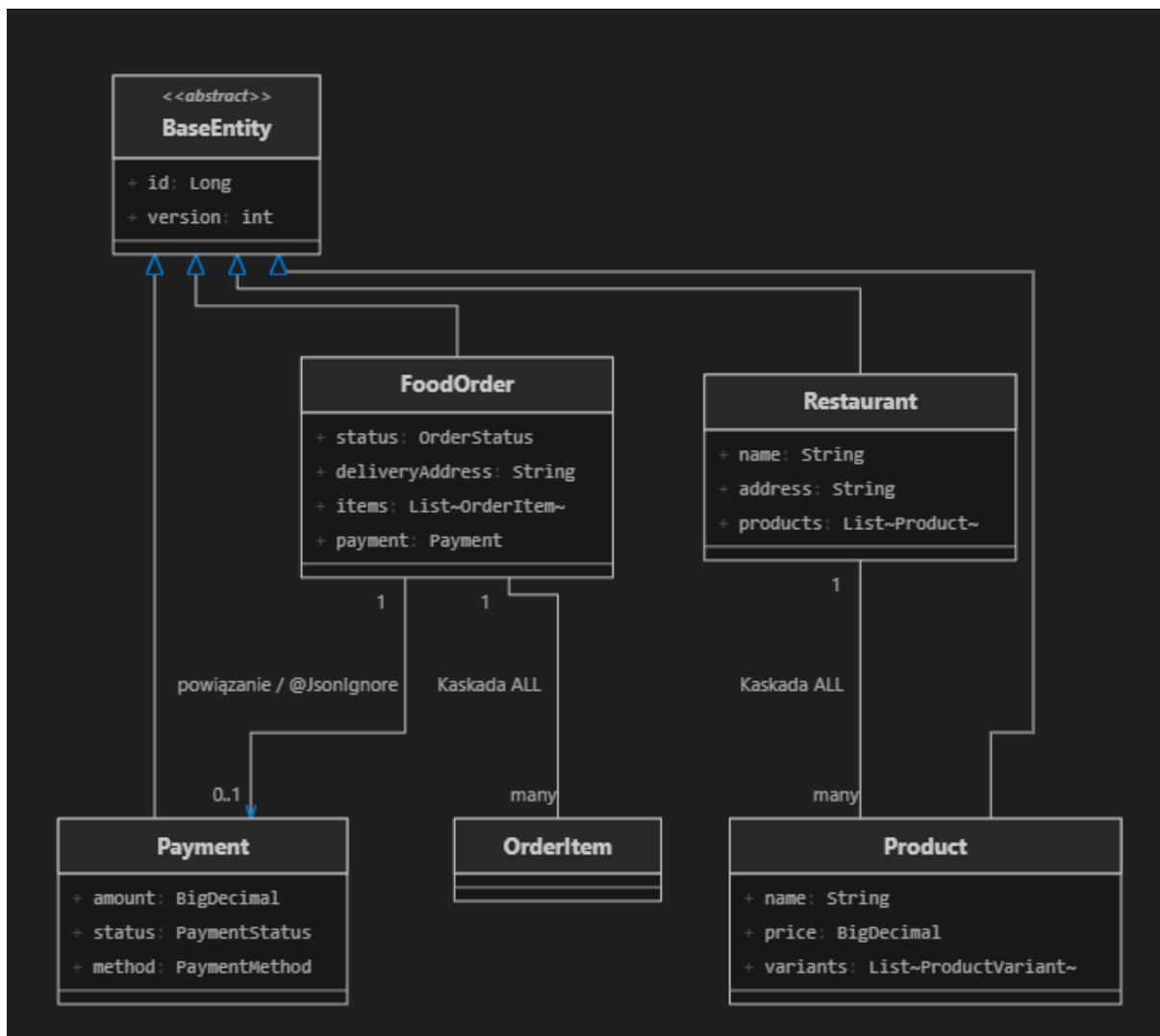
- **OrderItem (Encja wewnętrzna):** Reprezentuje wybrane danie w zamówieniu. Przechowuje zamrożone wartości `price` oraz `productName` z momentu zakupu, co zabezpiecza historię zamówień przed zmianami cen w menu restauracji.

5.2.4. Agregat Payment (Rozliczenia Finansowe)

Agregat ten odpowiada za transakcje płatnicze w systemie.

- **Payment (Korzeń Agregatu):** Zarządza transakcją (kwota, status, metoda płatności). Posiada relację `@OneToOne` z zamówieniem.
- **FoodOrder (Encja powiązana):** Reprezentuje powiązane zamówienie. W klasie `Payment.java` relacja zwrotna została oznaczona adnotacją `@JsonIgnore` (rozwiązanie cykliczności), aby uniknąć zapętlenia biblioteki Jackson podczas serializacji do formatu JSON.

5.2.5. Diagram Klas



4.3. Logika Biznesowa i Przypadki użycia

5.3.1. Przebieg Przypadków Użycia

Implementacja w warstwie serwisowej w pełni pokrywa następujące przypadki użycia zdefiniowane w wymaganiach systemu:

- **UC2: *Składanie zamówienia***: Metoda `placeOrder` w `FoodOrderService.java` tworzy nowe zamówienie w stanie `PENDING`, rejestruje czas jego złożenia (`orderTime`) oraz wylicza szacowany czas dostawy (`estimatedDeliveryTime = orderTime + 45 minut`).
- **UC3: *Oplacenie zamówienia***: Metoda `payOrder` wiąże transakcję finansową z zamówieniem. Dla płatności `ONLINE` status transakcji zmienia się natychmiast na `COMPLETED`, a zamówienia na `ACCEPTED`. Dla metody `ON_DELIVERY` (płatność przy odbiorze) transakcja pozostaje w stanie oczekiwania (`PENDING`).
- **UC4: *Śledzenie statusu zamówienia***: Endpoint `/food-orders/{id}/status` pozwala na bieżąco pobierać uproszczony podgląd zamówienia (aktualny stan, adres oraz planowany czas dostawy).
- **UC6: *Przeglądanie historii zamówień***: Metoda `getOrdersByCustomerId` pobiera z bazy i zwraca listę wszystkich historycznych zamówień powiązanych z danym klientem.
- **UC8: *Odbiór zamówienia z lokalu***: Metoda `pickupOrder` zmienia status zamówienia na `IN_DELIVERY`, potwierdzając, że kurier odebrał jedzenie z restauracji i ruszył w trasę.
- **UC9: *Dostarczenie zamówienia do klienta***: Metoda `deliverOrder` finalizuje cały proces dostawy, zmieniając status zamówienia na `DELIVERED`.
- **UC10: *Dodanie restauracji/sklepu***: Endpointy w `RestaurantController.java` umożliwiają administratorom rejestrację nowych lokali w systemie.
- **UC11: *Zarządzanie ofertą produktową***: Metody w `RestaurantService` obsługują dodawanie nowych dań do menu oraz usuwanie produktów z oferty z automatycznym czyszczeniem bazy danych (dzięki mechanizmowi `orphanRemoval`).
- **UC12: *Przypisanie kuriera do dostawy***: Podczas wywołania odbioru zamówienia (`pickupOrder`), system wymaga podania identyfikatora kuriera i zapisuje go w szczegółach dostawy.
- **UC13: *Rejestrowanie czasów dostaw***: System automatycznie rejestruje w bazie danych dokładną datę i godzinę dostarczenia (`deliveryTime`) w momencie wywołania zamknięcia zamówienia.

- **Automatyczne rozliczenie płatności przy odbiorze:** Dodatkowa logika biznesowa, która w momencie dostarczenia zamówienia (DELIVERED) automatycznie zmienia powiązaną płatność pobraniową z PENDING na COMPLETED.

5.3.2. Biznesowe Maszyny Stanów

W celu wyeliminowania błędów logicznych, zmiana statusów zamówień i płatności podlega ścisłej kontroli przejść stanów w serwisach:

Tabela 4.1. Biznesowe Maszyny Stanów

Status bieżący		Dozwolone przejścia	Typ stanu
PENDING	—▶	ACCEPTED / CANCELLED	Pośredni
ACCEPTED	—▶	IN_PREPARATION / IN_DELIVERY / CANCELLED	Pośredni
IN_PREPARATION	—▶	READY_FOR_PICKUP / IN_DELIVERY / CANCELLED	Pośredni
READY_FOR_PICKUP	—▶	IN_DELIVERY / CANCELLED	Pośredni
IN_DELIVERY	—▶	DELIVERED / CANCELLED	Pośredni
DELIVERED ★	—▶	Brak możliwości dalszych zmian statusu	Stan końcowy
CANCELLED ★	—▶	Brak możliwości dalszych zmian statusu	Stan końcowy

Próba wykonania niedozwolonego przejścia (np. ponowne otwarcie dostarczonego zamówienia) rzuca wyjątek `ConflictException`, który jest automatycznie mapowany na kod HTTP 409 Conflict.

5.3.3. Walidacja danych wejściowych

Przed uruchomieniem jakiejkolwiek logiki biznesowej, dane wejściowe w klasach Request DTO są dokładnie weryfikowane przez Spring Boot Validation. Reguły te zostały zaimplementowane w wielu plikach żądań w systemie:

- **@NotBlank (brak pustych tekstów):** Wdrożona na polach takich jak nazwa, adres i kontakt restauracji w `CreateRestaurantRequest.java` oraz adres dostawy w `CreateFoodOrderRequest.java`

- **@NotNull (wymagalność wartości):** Stosowana dla kluczowych powiązań – np. identyfikatora klienta czy restauracji podczas składania zamówienia.
- **@Positive (wartości dodatnie):** Wykorzystywana do weryfikacji kwot finansowych w celu zablokowania możliwości ustawienia ujemnej ceny produktu w menu restauracji.
- **@Valid (walidacja kaskadowa):** Wymusza sprawdzenie poprawności zagnieżdżonych struktur (np. weryfikuje każdy produkt w koszyku zamówienia lub każdy wariant dodawanego produktu).

Poniżej przedstawiono przykład implementację walidacji danych wejściowych w klasie żądania rejestracji restauracji.

```

1  package edu.prz.delivery.restaurants.domain.restaurant.dto;
2
3  import io.swagger.v3.oas.annotations.media.Schema;
4  import jakarta.validation.constraints.NotBlank;
5  import lombok.AllArgsConstructor;
6  import lombok.Builder;
7  import lombok.Data;
8  import lombok.NoArgsConstructor;
9
10 @Data
11 @Builder
12 @NoArgsConstructor
13 @AllArgsConstructor
14 public class CreateRestaurantRequest {
15
16     @NotBlank(message = "Nazwa restauracji nie może być pusta")
17     @Schema(example = "Pizzeria Bella Italia")
18     private String name;
19
20     @NotBlank(message = "Adres restauracji nie może być pusty")
21     @Schema(example = "Krakowskie Przedmieście 12, Warszawa")
22     private String address;
23
24     @NotBlank(message = "Godziny otwarcia są wymagane")
25     @Schema(example = "12:00 - 23:00")
26     private String openingHours;
27
28     @NotBlank(message = "Dane kontaktowe są wymagane")
29     @Schema(example = "22-123-45-67")
30     private String contactInfo;
31 }

```

4.4. Warstwa REST API (Endpointy)

Komunikacja z systemem odbywa się za pośrednictwem dedykowanych kontrolerów REST. Kontrolery operują wyłącznie na obiektach DTO (Data Transfer Objects), izolując model bazodanowy od warstwy prezentacji.

5.4.1. Wykaz wszystkich Endpointów (Swagger UI)

Poniżej przedstawiono kontrolery:

- Restauracji: *restaurant-controller*

Metoda	Ścieżka (URI)	Opis funkcjonalności
GET	/restaurants	Pobranie listy wszystkich restauracji
POST	/restaurants	Rejestracja nowego lokalu w systemie
GET	/restaurants/{id}	Pobranie szczegółowych danych restauracji po ID
PUT	/restaurants/{id}	Aktualizacja danych adresowych/kontaktowych restauracji
DELETE	/restaurants/{id}	Usunięcie restauracji z bazy danych
GET	/restaurants/{id}/products	Pobranie aktualnego menu wybranej restauracji
POST	/restaurants/{id}/products	Dodanie nowego dania do menu restauracji
DELETE	/restaurants/{id}/products/{productId}	Usunięcie dania z menu restauracji (kaskadowo)

- Płatności: *payment-controller*

Tabela 4.2. Kontroler Płatności

Metoda	Ścieżka (URI)	Opis funkcjonalności
GET	/payments	Pobranie listy wszystkich płatności
POST	/payments	Ręczne utworzenie nowej płatności w bazie
GET	/payments/{id}	Pobranie szczegółów płatności po ID
PUT	/payments/{id}	Aktualizacja danych płatności
DELETE	/payments/{id}	Usunięcie rekordu płatności
PATCH	/payments/{id}/status	Bezpośrednia aktualizacja statusu płatności (weryfikacja maszyny stanów)
GET	/payments/order/{orderId}	Szybkie wyszukanie płatności powiązanej z zamówieniem

- Zamówień: *food-order-controller*

Metoda	Ścieżka (URI)	Opis funkcjonalności
GET	/food-orders	Pobranie listy wszystkich złożonych zamówień
POST	/food-orders	Składanie nowego zamówienia (status: PENDING)
GET	/food-orders/{id}	Pobranie szczegółów wybranego zamówienia po ID
PUT	/food-orders/{id}	Aktualizacja danych zamówienia
DELETE	/food-orders/{id}	Usunięcie zamówienia z bazy danych
POST	/food-orders/{id}/pay	Opłacenie zamówienia (online lub przy odbiorze)
POST	/food-orders/{id}/pickup	Przypisanie kuriera i odbiór z restauracji (IN_DELIVERY)
POST	/food-orders/{id}/deliver	Potwierdzenie dostarczenia do klienta (DELIVERED)
GET	/food-orders/{id}/status	Pobranie skróconego statusu i czasów dostawy
PATCH	/food-orders/{id}/status	Bezpośrednia zmiana statusu zamówienia (maszyna stanów)
GET	/food-orders/customer/{customerId}	Pobranie historii zamówień konkretnego klienta

5.4.2. Spójne obsługiwane kodów błędów

Wszystkie błędy generowane przez endpointy są przechwytywane przez `GlobalExceptionHandler.java`, który tłumaczy je na ujednolicony format JSON zwracający:

1. 400 Bad Request przy błędach walidacji żądania (z precyzyjną listą niepoprawnych pól).
2. 404 Not Found przy odwołaniu do nieistniejących identyfikatorów.
3. 409 Conflict przy naruszeniu reguł przejść stanów w logice biznesowej.

```

GlobalExcept...Handler.java
1 > ...
14
15 @RestControllerAdvice
16 public class GlobalExceptionHandler {
17
18     @ExceptionHandler(ResourceNotFoundException.class)
19     public ResponseEntity<ErrorResponse> handleResourceNotFoundException(ResourceNotFoundException ex) {
20         ErrorResponse error = ErrorResponse.builder()
21             .message(ex.getMessage())
22             .status(HttpStatus.NOT_FOUND.value())
23             .timestamp(Instant.now().toString())
24             .build();
25         return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
26     }
27
28     @ExceptionHandler(ConflictException.class)
29     public ResponseEntity<ErrorResponse> handleConflictException(ConflictException ex) {
30         ErrorResponse error = ErrorResponse.builder()
31             .message(ex.getMessage())
32             .status(HttpStatus.CONFLICT.value())
33             .timestamp(Instant.now().toString())
34             .build();
35         return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
36     }
37

```

```

38     @ExceptionHandler(MethodArgumentNotValidException.class)
39     public ResponseEntity<ErrorResponse> handleValidationExceptions(MethodArgumentNotValidException ex) {
40         String validationDetails = ex.getBindingResult().getAllErrors().stream()
41             .map(error -> {
42                 String fieldName = ((FieldError) error).getField();
43                 String errorMessage = error.getDefaultMessage();
44                 return fieldName + ": " + errorMessage;
45             })
46             .collect(Collectors.joining(", "));
47
48         ErrorResponse error = ErrorResponse.builder()
49             .message("Błąd walidacji: [" + validationDetails + "]")
50             .status(HttpStatus.BAD_REQUEST.value())
51             .timestamp(Instant.now().toString())
52             .build();
53         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
54     }
55
56     @ExceptionHandler(IllegalArgumentException.class)
57     public ResponseEntity<ErrorResponse> handleIllegalArgumentException(IllegalArgumentException ex) {
58         ErrorResponse error = ErrorResponse.builder()
59             .message(ex.getMessage())
60             .status(HttpStatus.BAD_REQUEST.value())
61             .timestamp(Instant.now().toString())
62             .build();
63         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
64     }
65 }

```

4.5. Uruchomienie bazy danych H2, Seeder danych oraz dokumentacja Swagger

5.5.1. Baza danych H2 oraz automatyczna inizjalizacja (Database Seeding)

Aplikacja wykorzystuje wbudowaną bazę danych H2 pracującą w pamięci RAM (In-Memory Database), co eliminuje potrzebę instalowania zewnętrznych systemów zarządzania bazą danych podczas prac deweloperskich i prezentacji projektu.

Aby system był od razu gotowy do testów, wdrożyliśmy mechanizm automatycznego zasilania bazy danych (seeding). W klasie startowej `DeliveryApplication.java`

zaimplementowano komponent `CommandLineRunner`, który przy każdym uruchomieniu aplikacji sprawdza stan bazy. Jeśli baza jest pusta, automatycznie tworzy:

- 6 spolonizowanych restauracji o różnorodnej ofercie (Pizzeria, Burger House, Sushi, Tacos itp.) wraz z kompletnym menu i wariantami cenowymi dań.
- 3 przykładowe zamówienia w różnych stanach (oczekujące `PENDING`, przyjęte `ACCEPTED` oraz dostarczone `DELIVERED` z zarejestrowanym czasem dostawy i rozliczoną płatnością).

5.5.2. Swagger UI

Interaktywny panel Swagger UI jest dostępny pod adresem: <http://localhost:8080/swagger-ui/index.html>. Umożliwia on:

- Przeglądanie wszystkich dostępnych punktów końcowych (endpointów) podzielonych na kontrolery.
- Testowanie zapytań HTTP na żywo (opcja "Try it out") bezpośrednio z poziomu przeglądarki, bez potrzeby używania zewnętrznych narzędzi typu Postman.
- Wgląd w modele struktur przesyłanych danych (Request/Response DTO) wraz z przykładowymi wartościami pól.

5. Testowanie systemu

5.1. Cel testów

Celem testów było zweryfikowanie poprawności działania usług REST zaimplementowanych w systemie obsługi zamówień z dostawą. Testy zostały wykonane przy użyciu narzędzia Bruno, które służy do testowania usług REST API. Umożliwia tworzenie i wykonywanie zapytań HTTP, analizowanie odpowiedzi serwera oraz weryfikację poprawności działania poszczególnych endpointów aplikacji. Narzędzie Bruno zostało wykorzystane do realizacji wszystkich przypadków testowych oraz dokumentowania uzyskanych wyników.

Testy obejmowały zarówno scenariusze pozytywne, jak i negatywne. Weryfikowano poprawność działania endpointów REST API, walidację danych wejściowych, obsługę błędów oraz zgodność zwracanych odpowiedzi z założeniami projektowymi.

Aplikacja była uruchamiana lokalnie pod adresem: `http://localhost:8080`.

5.2. Metodyka testowania

Przypadki testowe zostały podzielone na trzy grupy:

- **Testy pozytywne:**

Ich celem było sprawdzenie poprawnego działania systemu dla prawidłowych danych wejściowych. Weryfikowano, czy aplikacja realizuje funkcjonalności zgodnie z wymaganiami oraz zwraca oczekiwane odpowiedzi.

- **Testy negatywne:**

Ich celem było sprawdzenie zachowania systemu w przypadku niepoprawnych danych wejściowych, nieistniejących identyfikatorów oraz prób naruszenia reguł biznesowych. Testy te pozwoliły zweryfikować poprawność mechanizmów walidacji oraz obsługi błędów.

- **Testy procesowe:**

Ich celem było sprawdzenie poprawności realizacji pełnych procesów biznesowych zachodzących w systemie. Weryfikowano poprawność realizacji pełnych procesów biznesowych, kolejność wykonywanych operacji oraz poprawność zmian statusów poszczególnych obiektów.

Każdy przypadek testowy zawierał:

- nazwę testu,
- endpoint,
- metodę HTTP,
- dane wejściowe,
- oczekiwany wynik,
- wynik rzeczywisty,
- status wykonania,
- dowód wykonania w postaci zrzutu ekranu.

Wszystkie przypadki testowe zostały przygotowane i wykonane przy użyciu narzędzia Bruno, a wyniki zostały udokumentowane w postaci zrzutów ekranu przedstawiających wysłane żądania oraz odpowiedzi zwracane przez aplikację.

5.3. Testowanie agregatu Restaurants

W ramach testowania agregatu Restaurants przeprowadzono kompleksową weryfikację funkcjonalności związanych z zarządzaniem restauracjami oraz produktami przypisanymi do restauracji.

6.3.1. Testy pozytywne

W ramach testów pozytywnych wykonano następujące przypadki testowe:

- **RES-SMOKE-01 – Wylistowanie wszystkich restauracji:** Zweryfikowano poprawność pobierania listy wszystkich restauracji zapisanych w systemie.
- **RES-SMOKE-02 – Pobranie restauracji po identyfikatorze:** Sprawdzone możliwość wyszukania konkretnej restauracji na podstawie jej identyfikatora.
- **RES-POS-03 – Utworzenie nowej restauracji:** Zweryfikowano poprawność procesu dodawania nowej restauracji do systemu.
- **RES-POS-04 – Aktualizacja danych restauracji:** Sprawdzone możliwość modyfikacji danych istniejącej restauracji.
- **RES-SMOKE-05 – Wylistowanie produktów w danej restauracji:** Zweryfikowano poprawność pobierania produktów przypisanych do wybranej restauracji.
- **RES-POS-06 – Dodanie produktu do restauracji:** Sprawdzone możliwość rozszerzenia oferty restauracji o nowy produkt.

- **RES-POS-07 – Usunięcie produktu z restauracji:** Zweryfikowano możliwość usunięcia produktu z menu restauracji.
- **RES-POS-08 – Usunięcie restauracji:** Sprawdzono poprawność procesu usuwania restauracji z systemu.

6.3.2. Testy negatywne

W celu sprawdzenia poprawności walidacji danych oraz obsługi błędów wykonano następujące testy:

- **RES-NEG-01 – Pobranie nieistniejącej restauracji:** Zweryfikowano zachowanie systemu podczas próby pobrania restauracji o nieistniejącym identyfikatorze.
- **RES-NEG-02 – Utworzenie restauracji z pustą nazwą:** Sprawdzono poprawność walidacji wymaganego pola nazwy restauracji.
- **RES-NEG-03 – Utworzenie restauracji bez adresu:** Zweryfikowano reakcję systemu na brak wymaganych danych adresowych.
- **RES-NEG-04 – Aktualizacja nieistniejącej restauracji:** Sprawdzono obsługę próby modyfikacji restauracji, która nie istnieje w bazie danych.
- **RES-NEG-05 – Usunięcie nieistniejącej restauracji:** Zweryfikowano zachowanie systemu podczas próby usunięcia nieistniejącego obiektu.
- **RES-NEG-06 – Dodanie produktu bez nazwy:** Sprawdzono poprawność walidacji wymaganych danych produktu.
- **RES-NEG-07 – Dodanie produktu z ujemną ceną:** Zweryfikowano mechanizmy walidacji wartości liczbowych.
- **RES-NEG-08 – Dodanie produktu do nieistniejącej restauracji:** Sprawdzono reakcję systemu na próbę powiązania produktu z nieistniejącą restauracją.
- **RES-NEG-09 – Usunięcie nieistniejącego produktu:** Zweryfikowano obsługę błędów podczas usuwania produktu, którego nie ma w systemie.

Łącznie wykonano 17 przypadków testowych. Wszystkie testy zakończyły się wynikiem PASS, co potwierdziło poprawność działania funkcjonalności agregatu Restaurants oraz mechanizmów walidacji danych.

5.4. Testowanie agregatu Food Orders

W ramach testowania agregatu Food Orders przeprowadzono kompleksową weryfikację funkcjonalności związanych z obsługą zamówień składanych przez klientów oraz realizacją procesu dostawy.

6.4.1. Testy pozytywne

W ramach testów pozytywnych wykonano następujące przypadki testowe:

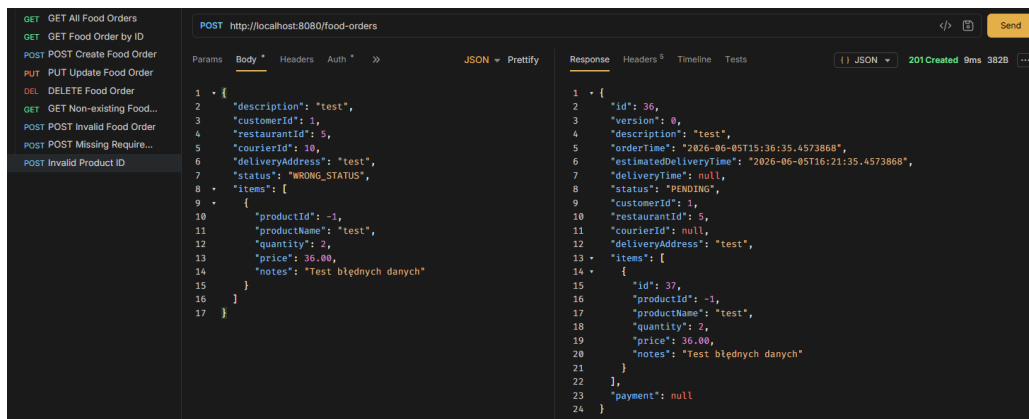
- **FO-POS-01 – Wylistowanie wszystkich zamówień:** Zweryfikowano możliwość pobrania listy wszystkich zamówień zapisanych w systemie.
- **FO-POS-02 – Pobranie zamówienia po identyfikatorze:** Sprawdzone możliwość wyszukania konkretnego zamówienia na podstawie jego identyfikatora.
- **FO-POS-03 – Utworzenie nowego zamówienia:** Zweryfikowano poprawność procesu tworzenia nowego zamówienia.
- **FO-POS-04 – Aktualizacja zamówienia:** Sprawdzone możliwość modyfikacji danych istniejącego zamówienia.
- **FO-POS-05 – Usunięcie zamówienia:** Zweryfikowano poprawność procesu usuwania zamówienia z systemu.

6.4.2. Testy negatywne

W celu sprawdzenia poprawności walidacji danych oraz obsługi błędów wykonano następujące testy:

- **FO-NEG-01 – Pobranie nieistniejącego zamówienia:** Zweryfikowano zachowanie systemu podczas próby pobrania zamówienia o nieistniejącym identyfikatorze.
- **FO-NEG-02 – Utworzenie zamówienia z niepoprawnymi danymi:** Sprawdzone reakcję systemu na przesłanie nieprawidłowych danych wejściowych.
- **FO-NEG-03 – Utworzenie zamówienia bez wymaganych pól:** Zweryfikowano poprawność mechanizmów walidacji wymaganych danych.
- **FO-NEG-04 – Utworzenie zamówienia z ujemnym identyfikatorem produktu.**

Test miał zweryfikować poprawność walidacji identyfikatora produktu podczas tworzenia zamówienia. Oczekiwano odrzucenia żądania i zwrócenia odpowiedniego błędu walidacji. W trakcie testów wykryto nieprawidłowość polegającą na zaakceptowaniu przez system ujemnego identyfikatora produktu, mimo że dane takie powinny zostać odrzucone przez mechanizmy walidacji.



Rys. 5.1. Wykryty błąd walidacji (FO-NEG-04)

6.4.3. Testy procesowe

Dodatkowo przeprowadzono testy procesowe mające na celu zweryfikowanie pełnego cyklu realizacji zamówienia:

- **FO-PROC-01 – Akceptacja zamówienia przez restaurację:** Zweryfikowano poprawność zmiany statusu zamówienia na ACCEPTED.
- **FO-PROC-02 – Realizacja płatności za zamówienie:** Sprawdzone poprawność procesu opłacenia zamówienia.
- **FO-PROC-03 – Odbiór zamówienia przez kuriera:** Zweryfikowano możliwość przekazania zamówienia do realizacji dostawy.
- **FO-PROC-04 – Dostarczenie zamówienia klientowi:** Sprawdzone poprawność zakończenia procesu realizacji zamówienia poprzez dostarczenie go do klienta.

Łącznie wykonano 13 przypadków testowych. Dwanaście testów zakończyło się wynikiem PASS, natomiast przypadek FO-NEG-04 zakończył się wynikiem FAIL, podczas którego wykryto błąd walidacji danych wejściowych. Wyniki testów potwierdziły poprawność działania funkcjonalności agregatu Food Orders oraz realizowanych procesów biznesowych.

5.5. Testowanie agregatu Payments

W ramach testowania agregatu Payments przeprowadzono kompleksową weryfikację funkcjonalności związanych z obsługą płatności powiązanych z zamówieniami.

6.5.1. Testy pozytywne

W ramach testów pozytywnych wykonano następujące przypadki testowe:

- **PAY-SMOKE-01 – Wylistowanie wszystkich płatności:** Zweryfikowano możliwość pobrania listy wszystkich płatności zapisanych w systemie.
- **PAY-SMOKE-02 – Pobranie płatności po identyfikatorze:** Sprawdzono możliwość wyszukania konkretnej płatności na podstawie jej identyfikatora.
- **PAY-SMOKE-03 – Pobranie płatności po identyfikatorze zamówienia:** Zweryfikowano możliwość pobrania płatności przypisanej do wskazanego zamówienia.
- **PAY-POS-04 – Aktualizacja płatności:** Sprawdzono możliwość modyfikacji danych istniejącej płatności.
- **PAY-POS-05 – Usunięcie płatności:** Zweryfikowano poprawność procesu usuwania płatności z systemu.
- **PAY-POS-06 – Aktualizacja statusu płatności:** Sprawdzono możliwość zmiany statusu płatności.

6.5.2. Testy negatywne

W celu sprawdzenia poprawności walidacji danych oraz obsługi błędów wykonano następujące testy:

- **PAY-NEG-01 – Pobranie nieistniejącej płatności:** Zweryfikowano zachowanie systemu podczas próby pobrania płatności o nieistniejącym identyfikatorze.
- **PAY-NEG-02 – Pobranie płatności dla nieistniejącego zamówienia:** Sprawdzono reakcję systemu podczas próby pobrania płatności przypisanej do nieistniejącego zamówienia.
- **PAY-NEG-03 – Utworzenie płatności dla zamówienia posiadającego już płatność:** Zweryfikowano poprawność działania reguł biznesowych uniemożliwiających przypisanie więcej niż jednej płatności do tego samego zamówienia.
- **PAY-NEG-04 – Aktualizacja płatności z istniejącym orderId:** Sprawdzono zachowanie systemu podczas próby przypisania płatności do zamówienia, które posiada już przypisaną płatność.
- **PAY-NEG-05 – Usunięcie nieistniejącej płatności:** Zweryfikowano obsługę błędu podczas próby usunięcia płatności, która nie istnieje w systemie.
- **PAY-NEG-06 – Aktualizacja nieistniejącej płatności:** Zweryfikowano obsługę błędu podczas próby aktualizacji płatności o nieistniejącym identyfikatorze.

- **PAY-NEG-07 – Utworzenie płatności dla nieistniejącego zamówienia:**
Zweryfikowano poprawność walidacji relacji pomiędzy płatnością a zamówieniem.
- **PAY-NEG-08 – Aktualizacja statusu nieistniejącej płatności:** Sprawdzono zachowanie systemu podczas próby zmiany statusu nieistniejącej płatności.

Łącznie wykonano 14 przypadków testowych. Wszystkie testy zakończyły się wynikiem PASS, co potwierdziło poprawność działania funkcjonalności agregatu Payments oraz mechanizmów odpowiedzialnych za integralność danych pomiędzy zamówieniami i płatnościami. Testy potwierdziły również prawidłową obsługę błędów dla nieistniejących zasobów oraz sytuacji naruszających reguły biznesowe systemu.

5.6. Podsumowanie wyników testów

W ramach procesu testowania przeprowadzono kompleksową weryfikację funkcjonalności systemu obsługi zamówień z dostawą. Testami objęto trzy główne agregaty systemu: Restaurants, Food Orders oraz Payments. Sprawdzono zarówno poprawność działania podstawowych funkcjonalności, jak i obsługę sytuacji wyjątkowych oraz pełnych procesów biznesowych. Łącznie wykonano 44 przypadki testowe, w tym:

Tabela 5.1. Przypadki testowe

Rodzaj testów	Liczba
Testy pozytywne	19
Testy negatywne	21
Testy procesowe	4
Razem	44

Uzyskane wyniki przedstawiają się następująco:

Tabela 5.2. Wyniki testów

Status	Liczba
PASS	43
FAIL	1
Razem	44

Zweryfikowano poprawność działania operacji CRUD, mechanizmów walidacji danych wejściowych, obsługi błędów oraz procesów biznesowych związanych z realizacją zamówień, obsługą restauracji oraz płatności.

Podczas testów wykryto jeden błąd funkcjonalny w agregacie Food Orders. System umożliwiał utworzenie zamówienia zawierającego ujemny identyfikator produktu, mimo że takie dane powinny zostać odrzucone przez mechanizmy walidacji. Wykryta nieprawidłowość wskazuje na brak pełnej walidacji danych wejściowych podczas tworzenia zamówienia. Przypadek został oznaczony statusem FAIL.

5.7. Wnioski

Przeprowadzone testy pozwoliły na ocenę jakości zaimplementowanych usług REST oraz stopnia realizacji wymagań funkcjonalnych systemu. Weryfikacja objęła zarówno standardowe scenariusze użytkowania aplikacji, jak i sytuacje związane z niepoprawnymi danymi wejściowymi oraz naruszeniem reguł biznesowych.

Największą liczbę przypadków testowych wykonano dla agregatów Restaurants oraz Payments, co umożliwiło szczegółową analizę funkcjonalności związanych z zarządzaniem restauracjami, produktami i płatnościami. Testy potwierdziły poprawność działania większości endpointów oraz prawidłową obsługę błędów i wyjątków zwracanych przez system.

W agregacie Food Orders wykryto pojedynczy błąd związany z niewystarczającą walidacją danych wejściowych podczas tworzenia zamówienia. System zaakceptował ujemny identyfikator produktu, mimo że takie dane powinny zostać odrzucone. Wykryta nieprawidłowość została udokumentowana.

Na podstawie uzyskanych wyników można stwierdzić, że aplikacja spełnia większość założonych wymagań funkcjonalnych. Pomimo wykrycia jednego błędu, pozostałe funkcjonalności działały zgodnie z oczekiwaniami, a przeprowadzone testy potwierdziły poprawność implementacji kluczowych procesów biznesowych. System może stanowić solidną podstawę do dalszego rozwoju i rozbudowy o kolejne funkcjonalności.

6. Role w projekcie

Tabela 6.1. Role w projekcie

Imię i nazwisko	Nr. indeksu	Zespół	Zakres obowiązków
Maria Małysa	176833	Analitycy	- opracowanie schematu diagramu ERD, - poszukiwanie przypadków użycia na zajęciach.
Martyna Marcinik	176834		- opracowanie opisu wymagań, - zarządzanie repozytorium, - obsługa kwestii technicznych związanych z GitHub, - napisanie wprowadzenia i podsumowania do sprawozdania, - formatowanie pliku sprawozdania.
Kacper Lis	176828		- opracowanie głównego diagramu Przypadków Użycia oraz pakietu szczegółowych diagramów z podziałem na poszczególne konteksty systemowe, - identyfikacja i poszukiwanie przypadków użycia na zajęciach.
Piotr Łukasik	176831		- sporządzenie opisu głównych funkcjonalności aplikacji, - identyfikacja przypadków użycia na zajęciach.
Julia Łój	176829	Projektanci	- opracowanie diagramu ERD dla kontekstów Zamówienie i Dostarczanie.
Dominika Łuczejko	176830		- opracowanie diagramu ERD dla kontekstów: Autentykacja, Marketing, Reklamacje, Restauracje.
Nina Magdoń	176832		- opracowanie diagramu ERD dla kontekstów Oferta i Płatność.

Damian Niemczyk	176838	Deweloperzy	- zaprojektowanie i wdrożenie agregatów oraz encji bazodanowych dla restauracji, zamówień i płatności.
Mateusz Mucha	176837		- zaimplementowanie logiki biznesowej dla wybranych przypadków użycia.
Hubert Pas	176845		- wdrożenie logiki zmiany statusów oraz walidacji danych.
			- stworzenie REST API z odpowiednimi endpointami do obsługi wszystkich operacji i komunikacji z bazą.
Wiktoria Nyklewicz	176842	Testerzy	- opracowanie przypadków testowych dla agregatu Payments, - wykonanie testów REST API w Bruno, analiza wyników.
Wojciech Kyc	176825		- ustalenie z grupą Developerów wymagań i poprawek umożliwiających pracę programistom, - stworzenie struktury folderu Testcase, - opracowanie przypadków testowych dla agregatu Restaurants, - wykonanie testów REST API w aplikacji Bruno, - udokumentowanie i analiza wyników.
Bartosz Nowak	176840		- opracowanie przypadków testowych dla agregatu Food-Orders, - wykonanie testów REST API w Bruno, - udokumentowanie i analiza wyników.

7. Podsumowanie

Prace nad systemem obsługi zamówień z dostawą pozwoliły w praktyce przejść przez wszystkie etapy tworzenia nowoczesnego oprogramowania – od wstępnej analizy biznesowej, przez projektowanie architektury i pisanie kodu, aż po końcowe testy. Zastosowanie podejścia Domain-Driven Design (DDD) okazało się dobrą decyzją. Dzięki temu został stworzony przejrzysty kod, w którym skomplikowane reguły biznesowe (takie jak maszyny stanów dla zamówień) zostały skutecznie zabezpieczone i oddzielone od warstwy technicznej.

Zbudowana aplikacja backendowa działa bardzo stabilnie, co potwierdziły testy wykonane w narzędziu Bruno. Na 44 przeprowadzone scenariusze testowe (zarówno pozytywne, negatywne, jak i procesowe), aż 43 zakończyły się sukcesem. Udało się wyłapać tylko jedną drobną niedoskonałość związaną z brakiem walidacji ujemnego identyfikatora produktu. Pozostałe mechanizmy – od składania zamówień, przez płatności, aż po przypisywanie kurierów – funkcjonują dokładnie tak, jak zaplanowano.

Podział grupy na wyspecjalizowane role (analityków, projektantów, programistów i testerów) przyniósł bardzo dobre efekty i pozwoliło na płynną, równoległą pracę. Ostatecznie stworzono spójne i bezpieczne REST API. Była to okazja do nauki i świetne przygotowanie do pracy przy rzeczywistych, komercyjnych aplikacjach klasy Enterprise.

Spis rysunków

Rys. 2.1. Diagram klas - model ERD	7
Rys. 2.2. Diagram przypadków użycia - model ogólny systemu.....	10
Rys. 3.1. Podział na konteksty	11
Rys. 3.2. Kontekst Zamówienie	12
Rys. 3.3. Kontekst Dostarczenie	12
Rys. 3.4. Kontekst Oferta	13
Rys. 3.5. Kontekst Płatności	13
Rys. 3.6. Kontekst Autentykacji	14
Rys. 3.7. Kontekst Marketingu	14
Rys. 3.8. Kontekst Reklamacji.....	15
Rys. 3.9. Kontekst Restauracji.....	15
Rys. 3.10. Diagram przypadków użycia - kontekst Autentykacja	17
Rys. 3.11. Diagram przypadków użycia - kontekst Dostarczanie.....	18
Rys. 3.12. Diagram przypadków użycia - kontekst Oferta	19
Rys. 3.13. Diagram przypadków użycia - kontekst Płatności.....	20
Rys. 3.14. Diagram przypadków użycia - kontekst Reklamacje.....	21
Rys. 3.15. Diagram przypadków użycia - kontekst Restauracje.....	22
Rys. 3.16. Diagram przypadków użycia - kontekst Zamówienia	23
Rys. 5.1. Wykryty błąd walidacji (FO-NEG-04)	38

Spis tabel

Tabela 4.1. Biznesowe Maszyny Stanów	28
Tabela 4.2. Kontroler Płatności.....	30
Tabela 5.1. Przypadki testowe	40
Tabela 5.2. Wyniki testów	40
Tabela 6.1. Role w projekcie.....	42